ClassroomTwin - End-Semester Upgrade

Corrected, fully specified build plan for a Unity 3D digital twin with Mamdani fuzzy lighting control (IEEE CCIS 2024) and fuzzy smart win-curtain co-regulation (IEEE IoT-J 2026). Written so an AI coding model can build it in one pass.

Reviewed and rebuilt from the student roadmap "*Digital Twin Ecosystem: End-Semester Advanced Upgrade Specification*". Every number in this document was computed by an independent Python reference model and reproduced by the C# core (98/98 self-test checks pass, compiled with Mono). Date: 10 Nov 2026.

What you get	Where
Verdict on the original roadmap: 9 critical, 16 major defects. The plan's code would compile but would not implement Mamdani logic, would never open the blinds below 25°, and would print savings numbers that the code cannot produce.	Section 1
Every assumption, constant and equation, with units and justification.	Sections 2, 5
Exact geometry (positions, scales, names) for the 3D model.	Section 4
Exact fuzzy sets, all 27 + 18 rules, operators, discretisation and two hand-checkable worked examples.	Section 6
Exact scenario definitions S0-S6, step order and Unity runtime mapping.	Sections 7-9
Expected 24-h results for 3 weather profiles - the acceptance criteria.	Section 10
Build order, verification checklist, troubleshooting, viva script.	Sections 11-12
Complete source code of all 8 C# files + the Python reference model.	Appendices A, B

Contents

0. Prompt to give the AI coding model 3

1. Review of the original roadmap 3

2. Assumptions and constants 4

3. Architecture 5

4. 3D geometry specification 6

5. Physical and sensor models (exact equations) 6

- 5.1 Sun and daylight . . . 6
- 5.2 Artificial lighting, power, comfort . . . 7
- 5.3 Thermal and air quality . . . 7
- 5.4 Occupancy, PIR and cooling KPI . . . 7

6. Mamdani fuzzy inference specification 7

- 6.1 Engine . . . 7
- 6.2 Fuzzy sets . . . 8
- 6.3 Light FIS rule base - CCIS 2024 Table I (27 rules, verbatim) . . . 8
- 6.4 Curtain FIS rule base - IoT-J 2026 Table I (R1-R11) + extensions (R12-R18) . . . 9
- 6.5 Worked example A - Light FIS (hand-checkable) . . . 9
- 6.6 Worked example B - Curtain FIS . . . 10
- 6.7 Replication of CCIS 2024 Table II (paper time slots) . . . 10

7. Scenario definitions (exact) 10

8. One control step - exact order 10

9. Unity runtime layer 10

- 9.1 Stepping . . . 11
- 9.2 Visual mapping (cosmetic, never affects numbers) . . . 11
- 9.3 Keys . . . 11

9.4 HUD content . . . 11
10. Expected results - acceptance criteria 11
10.1 What the numbers say (use these in the report) . . . 12
11. Build order and verification 14
11.1 Troubleshooting . . . 15
12. Viva demonstration script and honest limitations 15
12.1 Script (about 4 minutes) . . . 15
12.2 Limitations to state before the examiner does . . . 15
Appendix A - Complete source code 16
Assets/Scripts/Core/FuzzyCore.cs . . . 16
Assets/Scripts/Core/TwinConfig.cs . . . 17
Assets/Scripts/Core/TwinControllers.cs . . . 18
Assets/Scripts/Core/TwinSimulator.cs . . . 20
Assets/Scripts/Core/TwinSelfTest.cs . . . 22
Assets/Scripts/SimulationEngine.cs . . . 24
Assets/Editor/ClassroomBuilder.cs . . . 27
Assets/Editor/SimulationBuilder.cs . . . 29
Appendix B - Python reference model 31

0. Prompt to give the AI coding model

Paste the block below as the first message, attach this PDF and the accompanying **ClassroomTwin_Source.zip**. The zip holds the exact files listed in Appendix A; code copied out of a PDF picks up wrapped lines, so the zip is authoritative.

```
You are a senior Unity 3D engineer. Implement the ClassroomTwin end-semester upgrade EXACTLY as
specified in the attached PDF "ClassroomTwin End-Semester Upgrade - Corrected Build Specification".

1. Project: Unity 2021.3 LTS or newer, Built-in Render Pipeline,
   Active Input Handling = "Input Manager (Old)" or "Both".
   Project root: /Users/cparesh/Desktop/minor project/ClassroomTwin
2. Create/replace exactly these 8 files with the content of ClassroomTwin_Source.zip (Appendix A):
     Assets/Scripts/Core/FuzzyCore.cs         Assets/Scripts/Core/TwinConfig.cs
     Assets/Scripts/Core/TwinControllers.cs   Assets/Scripts/Core/TwinSimulator.cs
     Assets/Scripts/Core/TwinSelfTest.cs      Assets/Scripts/SimulationEngine.cs
     Assets/Editor/ClassroomBuilder.cs        Assets/Editor/SimulationBuilder.cs
   Delete any other copy of a class named ClassroomBuilder, SimulationBuilder or SimulationEngine.
3. Do NOT change any constant, membership function, rule, step order or formula. Do NOT
   "simplify" the fuzzy engine, replace it with if/else, or switch the Core folder to float.
   Core files must not reference UnityEngine.
4. Follow Section 11 (build order) step by step. After every step run the listed check;
   stop and fix before moving on.
5. Acceptance: "Digital Twin > Run Self-Test" logs "SELF-TEST PASSED (98 checks)", and
   "Digital Twin > Run 24h Batch Evaluation" reproduces Tables 10.1-10.3 to the printed precision.
6. If a Unity API differs in my version, change only the Unity-facing call, never the
   behaviour, and tell me what you changed.
7. When done, list every file you touched and paste the Console output of the self-test
   and the batch.
```

1. Review of the original roadmap

The four features you chose (blinds, Mamdani core, co-regulation, HUD + stress test) are the right ones and map cleanly onto the two papers. The problem is the execution: the roadmap describes a Mamdani controller but the code does not contain one, several numbers in the expected-results table cannot be produced by the code, and the 3D scene cannot physically show shading because the sun comes from the wrong side of an open-topped room. Each defect below was reproduced; the fix is already in the code of Appendix A.

1.1 Critical defects (wrong behaviour or wrong numbers)

#	Defect in the roadmap (as written)	Consequence	Fix in this spec
C1	`TriangularMF` handles shoulders by dividing by 0.0001. For [0,0,45], μ(0) = min(0/0.0001, 1) = 0; for [60,100,100], μ(100) = 0.	The darkest and brightest inputs get zero membership - exactly the cases the controller exists for.	Trapezoid with explicit shoulder logic (Sec. 6.1).
C2	S5 is a weighted sum ($\mu_L \cdot 1 + \mu_M \cdot 0.5 + \mu_H \cdot 0.2$), unnormalised. S6 is if/else on T > 30 °C with a hard-coded $F_c = 0.35$.	No rules, no min/max inference, no defuzzification: it is not Mamdani. An examiner reading the code will see this immediately.	Generic Mamdani engine; CCIS Table I (27 rules) and IoT-J Table I (R1-R11) entered verbatim.
C3	$\theta = 25° + (1 - F_c) \cdot 50°$, claimed to give θ → 0° when $F_c \approx 1$.	Range is [25°, 75°]: the blinds can never fully open. Algorithm 1 also compares $F_c \in [0,1]$ with L_a in %.	Angle comes from the curtain-FIS centroid on [0°, 75°]; F_c becomes a measured comfort KPI.
C4	Sun rotation Euler(35, −110, 0) and no ceiling.	Light travels towards −Z, i.e. from the corridor side, and floods in from above. Blinds cast no shadow into the room - the headline visual cannot work.	Ceiling added; sun animated with azimuth −60°…+60° so it always travels towards +Z through the windows.
C5	Scenario code vs claimed savings: S1 code is ON 00:00-18:00; S3 and S4 are gated by occupancy.	S1 gives 25 %, not 62 %; S3 gated would give ~82 %, not 46 %; S4 gated is not 8 %.	Exact definitions (Sec. 7): S1 62.50 %, S3 47.69 %, S4 5.56 % - computed, not estimated.
C6	One cumulative energy counter for all scenarios.	After any key press the displayed saving mixes several scenarios and means nothing.	Counters reset on switch + deterministic 24-h batch (key B / menu).
C7	Expected S6 saving "75-82 %" vs S0.	Not computed; most of any saving vs S0 comes from switching lights off in empty hours (PIR), not from fuzzy logic or curtains.	Report vs S0 and vs S2 (PIR), plus cooling load and indoor temperature (Sec. 10).

#	Defect in the roadmap (as written)	Consequence	Fix in this spec
C8	Crisp inputs are not clamped to their universe.	Found while verifying: on a heatwave day T_{in} reaches 42 °C > 40 °C, every temperature set gives μ = 0, no rule fires and the lights switch off at noon.	Clamp every input to [min, max]; if no rule fires, hold the previous output.
C9	"Exterior daylight %" fed straight into the controller; blind angle changes nothing physical.	The co-regulation loop (curtain → indoor daylight → LED dimming) does not exist in the model.	Daylight-factor model × slat transmission τ(θ) per light row; thermal gain also scaled by τ(θ).

1.2 Major defects (scene, engineering, robustness)

#	Defect	Fix
M1	Bay centres ±3.85 m. Real openings are −5.55…−2.05, −1.75…+1.75, +2.05…+5.55 (centre ±3.80). Slats clip 5 cm into the end wall and leave a 5 cm gap at the pillar.	Bay centres ±3.80 m, slat length 3.46 m (2 cm clearance each side).
M2	Slats at Z = −4.40 sit inside the wall; depth 0.12 m < pitch 0.15 m, so coverage at 75° = 0.12·sin75°/0.15 = 77 % - "closed" still leaks 23 %.	Interior Z = −4.20; depth 0.17, pitch 0.16 ⇒ coverage 1.0 at 75°.
M3	Slats span Y 0.95-2.30; the aperture is 0.80-2.40, leaving 25 cm unshaded.	10 slats at Y = 0.88 + 0.16·i (0.88…2.32) tile 0.80…2.40 exactly.
M4	Desk centre Y = 0.38 with height 0.75: desks sink 9.5 cm into the floor (floor top is Y = 0.10).	Desk centre Y = 0.475.
M5	Glass material is created but never used.	Three glass panes, shadow casting off, no collider.
M6	Materials made with `new Material()` in an editor script, no Undo, scene not marked dirty.	Materials saved to Assets/Materials/Twin; Undo + MarkSceneDirty.
M7	12 spotlights in Forward rendering: Unity's default pixel-light count lets only a few render per-pixel; soft shadows on 12 spots are expensive.	pixelLightCount ≥ 16, ForcePixel, spot shadows off, sun shadows soft.
M8	`renderer.material` written every frame creates material instances (leak).	MaterialPropertyBlock.
M9	Servo speed: comment 35°/s, code 45°/s, text 30°/s; animation in real seconds feeds the results.	Model angle is deterministic; visual servo 40°/s is cosmetic only.
M10	Integration per frame with Time.deltaTime: results depend on frame rate.	Fixed 1-minute steps from an accumulator.
M11	HUD snaps the label to 0°/45°/75° while the angle is continuous.	Bands: <15° Open, <60° Partial, else Closed; numeric angle shown.
M12	AQI computed but never used; Algorithm 1 AQ threshold unused; comfort Eq. (14) not implemented.	AQI is a curtain-FIS input (R8, R9); comfort band, F_c and Eq. (12) utilisation are KPIs.
M13	The paper's curtain table has gaps: High light + Hot + Work has no rule (the heatwave case!), Low light in Work has none.	Documented extension rules R12-R18 + hold-on-no-activation.
M14	Home time slots (Sleep 0-10 h) used in a classroom: at 09:00 μ_{Sleep} = 0.2 fires the Closed rules during class.	Classroom slots; the paper slots are kept only for the Table II self-test.
M15	Membership parameters do not follow the papers' figures (e.g. CCIS Fig. 6(a) Low has a plateau to ~25 %).	Parameters read from CCIS Fig. 6 and Table I ranges (Sec. 6.2).
M16	`Input.GetKeyDown` throws if the project uses only the new Input System.	Player Settings → Active Input Handling = Old or Both.

Minor issues also fixed: `ClassroomBuilder` derived from MonoBehaviour for no reason; GUIStyles were allocated every OnGUI call; unused variables (`muHighTemp` branch, DIMMED constant in S6); the PIR had no hold time so lights flickered off between motion events; the 6.2 table quoted savings with no stated simulation window.

2. Assumptions and constants

Where the papers or the mid-sem model did not fix a value, the value below was chosen and justified. All live in `TwinConfig.cs`; changing one changes Table 10.1, so do not edit them during the build.

Quantity	Value	Basis / justification
Simulation step	1 simulated minute; 1440 steps/day	Fixed step makes results frame-rate independent; IoT-J Alg. 1 updates every 15 s, which is < 1 sim-minute at demo speed.
Demo speed	15 sim-min per real second (96 s/day); keys [ ] halve/double, 1-120	Mid-sem value kept.
Fixture power	65 W full, 34 W dimmed (=500 lux), 12 fixtures, 780 W baseline	Mid-sem / MDPI 2022 model.

Quantity	Value	Basis / justification
Artificial illuminance	E_{art} = 500·65/34 = 955.882 lux per row zone at 100 %; linear in level	Linear lumen vs power (IoT-J Eq. 8: power ∝ intensity).
Light rows	Window (Z = −2.8), Center (0.0), Corridor (+2.8); 4 fixtures each	Mid-sem layout.
Daylight factor (blinds open)	5.0 % / 2.0 % / 0.8 % for the three rows	Typical side-lit classroom: 1.6, 4.4 and 7.2 m from the glass.
Exterior diffuse illuminance	$E_{diff} = E_{max}·S(h)$; E_{max} 20 000 lux (Normal, Pollution), 25 000 lux (Heatwave)	Clear-day diffuse horizontal values; sunrise 06:00, sunset 18:00.
LDR scale	100 % = 1000 lux (clamped)	Puts the CCIS 'High' set (≥ 60 %) at ≥ 600 lux, just above the 500 lux target.
Slat optics	depth 0.17 m, pitch 0.16 m, slat transmittance 0.05	Coverage reaches 1 at 75° (IoT-J: 75° = fully closed).
Static blinds S0-S5	0° (horizontal = unshaded)	The mid-sem twin had bare windows; S0-S5 therefore see full daylight and full solar gain.
Occupancy	Occupied 09:00-13:00 and 14:00-18:00 (8 h); lunch 13:00-14:00 empty	Typical lecture day.
PIR hold	10 min after last occupancy	Common PIR time-out; avoids flicker.
Outdoor temperature	$T_{out} = T_{mean} + 4·\cos(2\pi(h - 15)/24)$; T_{mean} 24 °C (Normal), 34 °C (Heatwave)	Diurnal cycle, peak at 15:00 (Hyderabad-like).
Indoor temperature	First-order, τ = 1 h; solar rise up to 5 °C × S × τ(θ)	Fan-ventilated, non-AC classroom; makes blinds matter thermally.
Indoor AQI	First-order, τ = 0.5 h; equilibrium AQI_{out} + 60 when occupied; AQI_{out} 45 (Normal, Heatwave), 250 (Pollution)	CO_2/particulate build-up by occupants; Pollution profile exercises rules R8/R9.
Cooling-load KPI	SHGC 0.60, glazing 16.8 m², facade peak 400 / 550 W/m², COP 3.0, counted only when occupied	Estimate of AC electricity the room would need; reported, never fed back.
Comfort band	400 lux ≤ E_{task} ≤ 2000 lux on every row; $\mu_{comfort}$ = trap(300, 400, 2000, 3000)	400 lux = 0.8 × target; 2000 lux = useful-daylight upper limit.
Degree-hours	Σ max(0, T_{in} − 28 °C)·Δt while occupied	28 °C is a common adaptive-comfort upper limit.

3. Architecture

The single most important design decision: **all physics and control live in a pure C# core** (namespace `ClassroomTwin.Core`, no UnityEngine references, double precision). The MonoBehaviour only steps it and draws it. The live demo and the 24-h batch run the identical `Step()`, so what the examiner sees on screen and what you report in the table can never disagree. It also means the core can be unit-tested outside Unity (it was, with Mono).

File	Layer	Responsibility
Assets/Scripts/Core/FuzzyCore.cs	Core	FuzzySet (trapezoid/triangle), FuzzyVariable (clamping), FuzzyRule, MamdaniFIS (min-AND, max aggregation, clipped implication, discrete centroid, NaN when nothing fires).
Assets/Scripts/Core/TwinConfig.cs	Core	Every constant of Section 2, weather profiles, scenario names.
Assets/Scripts/Core/TwinControllers.cs	Core	Builds the Light FIS (CCIS 2024) and the Curtain FIS (IoT-J 2026) from the tables in Section 6.
Assets/Scripts/Core/TwinSimulator.cs	Core	Environment equations, one fixed step, scenario logic, KPI accumulation, 24-h batch, CSV table.
Assets/Scripts/Core/TwinSelfTest.cs	Core	98 checks generated from the Python reference (membership values, FIS outputs, 24-h energy).
Assets/Scripts/SimulationEngine.cs	Runtime	MonoBehaviour: fixed-step accumulator, slats/LEDs/sun mapping, keys, HUD, batch overlay.
Assets/Editor/ClassroomBuilder.cs	Editor	Menu 'Generate Classroom': geometry of Section 4, materials as assets.
Assets/Editor/SimulationBuilder.cs	Editor	Menus 'Add Simulation Engine' (+ camera), 'Run Self-Test', 'Run 24h Batch Evaluation'.

Data flow of one step (details in Section 8):

```
clock h ➔ sun S(h), T_out(h), occupancy, PIR ➔ window LDR ➔ [S6] Curtain FIS ➔ θ ➔ τ(θ)
τ(θ) ➔ daylight per row ➔ row LDR ➔ Light FIS ×3 ➔ LED level La[r] ➔ power, lux, comfort
```

```
τ(θ), S ↛ indoor temperature T_in (next step)    occupancy ↛ indoor AQI (next step)
```

The loop closes twice: the curtain changes the daylight the light controller sees (co-regulation), and the curtain changes the indoor temperature the curtain controller sees on the next step (thermal feedback).

4. 3D geometry specification

World units are metres, Y up. Floor slab centre Y = 0 (top at 0.10). Window wall is the −Z wall; the corridor is +Z; the blackboard is −X. All positions below are world positions of the object centre; scales are full sizes. Names are load-bearing: `SimulationEngine` finds objects by these names.

Object (name)	Position (x, y, z)	Scale (x, y, z)	Notes
Classroom_Room302	(0, 0, 0)	-	Root; regenerated on each menu run
Floor	(0, 0, 0)	(11.5, 0.2, 9.0)	Top surface Y = 0.10
Ceiling (new)	(0, 3.05, 0)	(11.5, 0.1, 9.0)	Bottom surface Y = 3.00
Wall_Back / Wall_Front	(±5.65, 1.5, 0)	(0.2, 3.0, 9.0)	Inner faces X = ±5.55
Wall_Corridor	(0, 1.5, 4.4)	(11.5, 3.0, 0.2)	
Wall_Window_Sill	(0, 0.4, −4.4)	(11.5, 0.8, 0.2)	Top Y = 0.80
Wall_Window_Header	(0, 2.7, −4.4)	(11.5, 0.6, 0.2)	Bottom Y = 2.40; window wall inner face Z = −4.30
Pillar_1 / Pillar_2	(∓1.9, 1.5, −4.4)	(0.3, 3.0, 0.25)	Span X ±(1.75…2.05)
Window_Glass / Glass_Bay_1..3 (new)	(−3.80 | 0 | 3.80, 1.6, −4.4)	(3.5, 1.6, 0.01)	Twin_Glass, no shadows, no collider
Teacher_Podium	(−4.7, 0.1, 0)	(1.5, 0.2, 6.0)	
Blackboard	(−5.5, 1.6, 0)	(0.05, 1.2, 4.5)	
Lighting_System / Row1_Window, Row2_Center, Row3_Corridor / LED_Fixture_1..4	x ∈ {−3.6, −1.2, 1.2, 3.6}, y = 2.95, z ∈ {−2.8, 0, 2.8}	(1.2, 0.05, 0.3)	Child 'Spotlight': local (0, −0.6, 0), rot (90, 0, 0), spot 75°, range 6, shadows none, ForcePixel
Student_Desks / Desk_x_z	x ∈ {−2, 0, 2, 4}, y = 0.475, z ∈ {−2, 0, 2}	(0.8, 0.75, 1.5)	12 desks, now resting on the floor
Sunlight	-	-	Directional, soft shadows; rotation driven at runtime (Sec. 9)
Smart_Blinds / Blinds_Bay_1..3	(−3.80 | 0 | 3.80, 0, −4.20)	(1, 1, 1)	Interior, 0.10 m in front of the wall face
Slat_b_s (b = 0..2, s = 0..9)	local (0, 0.88 + 0.16·s, 0)	(3.46, 0.012, 0.17)	Rotation = Euler(θ, 0, 0) about local X; no collider
Blind_Motor_Unit_1..3	local (−1.65, 2.44, 0)	(0.12, 0.06, 0.08)	MG996R + ZigBee end-device housing (visual)

4.1 Geometry checks (all verified)

- Clearances: slat half-depth projected on Z is ≤ 0.085 m, so the slat stays in Z ∈ [−4.285, −4.115], clear of the wall face (−4.30) and the glass (−4.40).
- At 75° the vertical half-extent is 0.085·sin75° = 0.0821 m: top slat reaches 2.402 m and bottom 0.798 m, both in front of (not inside) the header and sill.
- Neighbouring slats stay parallel; their perpendicular separation is 0.16·cos θ, which is 0.041 m at 75° - larger than the 0.012 m thickness, so slats never intersect.
- Unity: a positive rotation about local +X pitches the slat's room-side edge downwards. Either tilt direction blocks equally in this model.
- Shaded window area per bay = 3.5 × 1.6 = 5.6 m²; total glazing 16.8 m², which is the A_{win} used by the cooling KPI.

5. Physical and sensor models (exact equations)

Notation: k = step index (0…1439), h = k/60 is the hour at the start of the step, Δt = 1/60 h. Rows r = 0 (Window), 1 (Center), 2 (Corridor). Every equation is implemented once in `TwinSimulator`.

5.1 Sun and daylight

```
S(h) = sin(π·(h − 6)/12) for 6 ≤ h ≤ 18, else 0 S(9) = 0.70711, S(12) = 1
```

```
E_diff(h) = E_max · S(h)  E_max = 20 000 lux (Normal), 25 000 lux (Heatwave)
C(θ) = min(1, d·sinθ / p) = min(1, 1.0625·sinθ)  d = 0.17 m, p = 0.16 m
τ(θ) = 1 − C(θ)·(1 − 0.05)
```

θ	0°	25°	45°	75°
coverage C	0	0.44904	0.75128	1 (1.0263 clipped)
transmission τ	1.00000	0.57342	0.28626	0.05000

```
E_day[r] = DF[r] · E_diff · τ(θ)  DF = {0.050, 0.020, 0.008}
LDR_curtain = min(100, DF[0]·E_diff / 1000 · 100)  (unshaded window-plane potential, %)
LDR_row[r] = min(100, E_day[r] / 1000 · 100)  (what each row's light FIS sees, %)
```

At noon on a normal day with blinds open: E_{day} = 1000 / 400 / 160 lux; at 45° multiply by 0.28626; at 75° by 0.05.

5.2 Artificial lighting, power, comfort

```
E_art[r] = 955.882 · La[r]  La ∈ [0, 1]
P = Σ_r 4 · 65 W · La[r]  (P = 780 W when every La = 1)
E_task[r] = E_day[r] + E_art[r]
Fc(k) = (1/3)·Σ_r μ_comfort(E_task[r]),  μ_comfort = trap(300, 400, 2000, 3000)
in_band(k) = 1 if 400 ≤ E_task[r] ≤ 2000 for all r, else 0  (IoT-J Eq. 14 analogue)
U_avg = mean over occupied minutes of 100·(1/3)·Σ_r La[r]  Reduced artificial % = 100 − U_avg
(IoT-J Eq. 12)
```

5.3 Thermal and air quality

```
T_out(h) = T_mean + 4·cos(2π(h − 15)/24)  Normal: 20…28 °C, Heatwave: 30…38 °C
T_eq = T_out(h) + 5 °C · S(h) · τ(θ)
T_in(k+1) = T_in(k) + (T_eq − T_in(k)) · Δt / 1 h  T_in(0) = T_out(0)
AQI_eq = AQI_out + 60·occupied;  AQI(k+1) = AQI(k) + (AQI_eq − AQI(k)) · Δt / 0.5 h;  AQI(0) =
AQI_out
```

Explicit Euler is stable here because Δt/τ = 1/60 and 1/30, both far below 1.

5.4 Occupancy, PIR and cooling KPI

```
occupied(h) = (9 ≤ h < 13) or (14 ≤ h < 18)
PIR(k) = occupied(k) or (k − k_last_occupied ≤ 10) → lights stay on 09:00-13:10 and 14:00-18:10
Q_solar = 0.60 · 16.8 m² · I_peak · S(h) · τ(θ)  I_peak = 400 W/m² (Normal), 550 W/m² (Heatwave)
E_cool += (Q_solar + P) / 3.0 · Δt  (only while occupied; KPI only, not fed back)
```

6. Mamdani fuzzy inference specification

6.1 Engine

- Membership: trapezoid trap(a,b,c,d). μ = 0 if x < a or x > d; μ = 1 if b ≤ x ≤ c; rising edge (x − a)/(b − a), or 1 if a = b; falling edge (d − x)/(d − c), or 1 if c = d. Triangle tri(a,b,c) = trap(a,b,b,c).
- Every crisp input is first clamped to its variable's [min, max] (fixes C8).
- Rule strength α = min of the antecedent memberships; 'Any' contributes 1.
- Aggregation: activation of each output set = max α over the rules that conclude it.
- Implication: clip (min) the output set at its activation; union of clipped sets by max.
- Defuzzification: discrete centroid y* = Σ y·μ(y) / Σ μ(y) over y = U_{min} + i·0.5, i = 0 … round((U_{max} − U_{min})/0.5). Light output: [−10, 110] → 241 samples. Angle output: [−30, 105] → 271 samples.
- The output universes extend past the physical range so each fully-active symmetric triangle has its centroid exactly at its peak (20/50/80 %, 0°/45°/75° - the paper's three positions). The centroid is then clamped to [0, 100] % or [0°, 75°].
- If Σ μ ≤ 1e-12 (no rule fired) the engine returns NaN: the curtain holds its previous angle; the light falls back to 0 (never happens with the clamped inputs).
- Use double precision; iterate rules and samples in the listed order (it only affects the last bits).

6.2 Fuzzy sets

Variable (universe)	Sets: trapezoid (a, b, c, d) or triangle (a, b, c)	Source
Light input L [0, 100] %, both FIS	Low (0, 0, 25, 45) · Medium (35, 55, 75) · High (60, 80, 100, 100)	CCIS Fig. 6(a) + Table I ranges
Time slot, classroom [0, 24] h	Sleep (0, 0, 6, 8) · Work (7, 9, 17, 19) · Relax (17, 19, 24, 24)	Adapted to a teaching day
Time slot, paper [0, 24] h (self-test only)	Sleep (0, 0, 5, 10) · Work (8, 14, 20) · Relax (18, 20, 24, 24)	CCIS Fig. 6(b)
Temperature, Light FIS [0, 40] °C	Cold (0, 0, 10, 20) · Warm (15, 22.5, 30) · Hot (25, 30, 40, 40)	CCIS Fig. 6(c)
Temperature, Curtain FIS [0, 40] °C	Cold (0, 0, 16, 22) · Warm (18, 25, 32) · Hot (28, 34, 40, 40)	Shifted so 'Hot' means a hot classroom (≥ 28 °C)
AQI, Curtain FIS [0, 500]	Clean (0, 0, 100, 200) · Poor (100, 200, 500, 500)	IoT-J Table I uses Clean / Poor
Output: artificial level [−10, 110] → clamp [0, 100] %	Low (−10, 20, 50) · Medium (20, 50, 80) · High (50, 80, 110)	CCIS Table II: Low ≈ 20-28, Medium ≈ 55, High ≈ 69
Output and prev-angle input: θ [−30, 105] → clamp [0, 75] °	Open (−30, 0, 30) · Partial (15, 45, 75) · Closed (45, 75, 105)	IoT-J Fig. 3: 0°, 45°, 75°

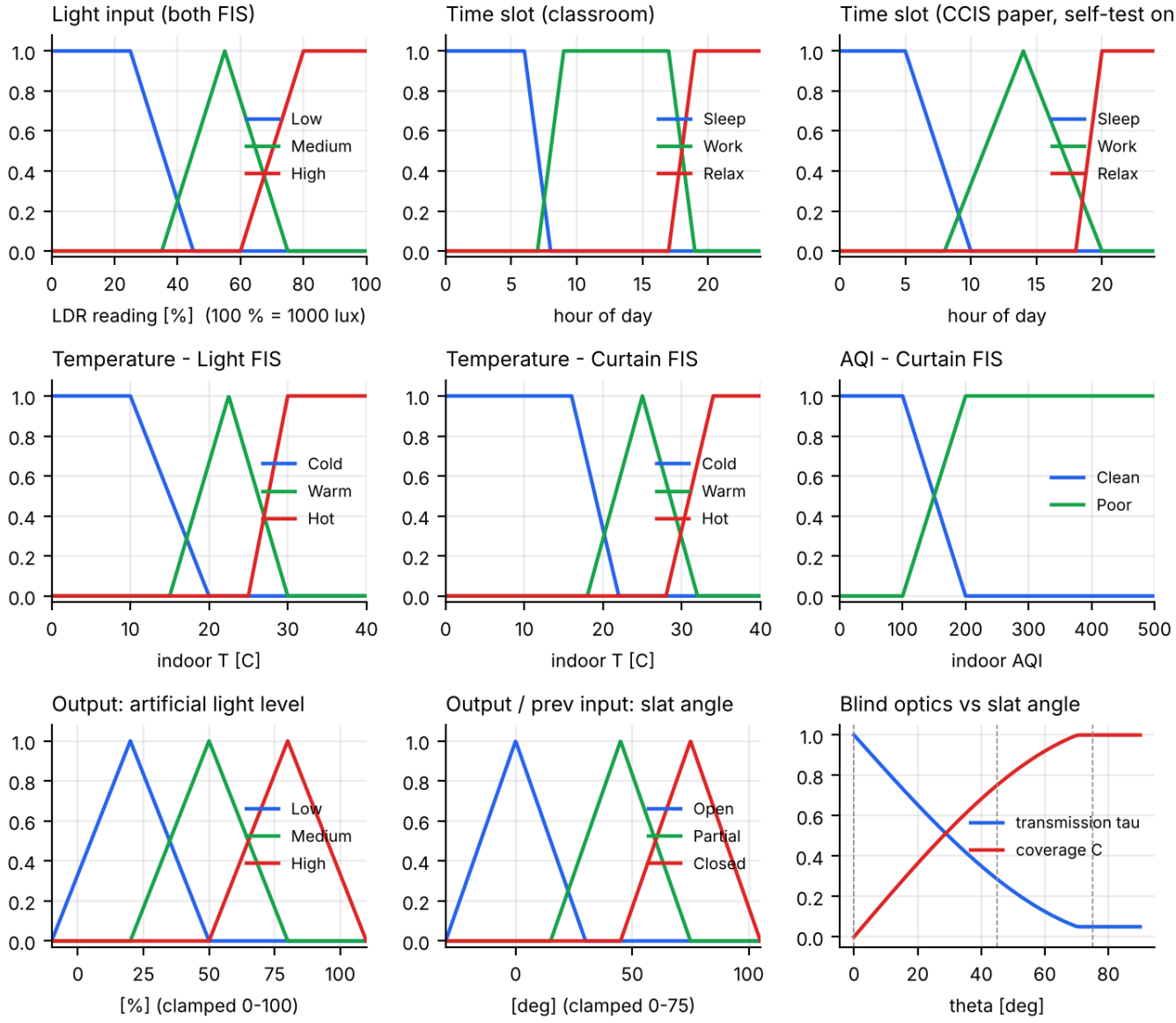


Figure 6.1 - every membership function used, plus slat coverage and transmission versus angle.

6.3 Light FIS rule base - CCIS 2024 Table I (27 rules, verbatim)

Inputs (L, Slot, T) → artificial level. Rule IDs L1…L27 run Light (Low, Medium, High) × Slot (Work, Relax, Sleep) × Temp (Cold, Warm, Hot) in that nesting order.

Light	Slot	Cold	Warm	Hot
Low	Work	High	Medium	Medium
Low	Relax	Medium	Medium	Low
Low	Sleep	Low	Low	Low
Medium	Work	High	High	Medium
Medium	Relax	Medium	Medium	Medium
Medium	Sleep	Low	Low	Low
High	Work	Medium	Low	Low
High	Relax	Medium	Medium	Low
High	Sleep	Medium	Low	Low

Note for the viva: the paper's table is not monotonic in light (Medium light + Work + Cold/Warm → High, brighter than Low light → Medium). It is implemented verbatim on purpose; it is why S5/S6 sometimes add light when daylight is moderate. Do not 'fix' it - cite it as an observation.

6.4 Curtain FIS rule base - IoT-J 2026 Table I (R1-R11) + extensions (R12-R18)

Rule	Light	Temp	AQI	Slot	Prev angle	→ New angle	Origin
R1	Low	Any	Any	Sleep	Any	Closed	IoT-J Table I
R2	Medium	Cold	Clean	Work	Open	Open	IoT-J Table I
R3	Medium	Warm	Clean	Work	Any	Partial	IoT-J Table I
R4	Medium	Any	Any	Relax	Any	Partial	IoT-J Table I
R5	High	Cold	Clean	Work	Open	Open	IoT-J Table I
R6	High	Warm	Clean	Work	Partial	Open	IoT-J Table I
R7	High	Hot	Clean	Relax	Any	Partial	IoT-J Table I
R8	High	Any	Poor	Any	Any	Partial	IoT-J Table I
R9	Medium	Any	Poor	Any	Any	Partial	IoT-J Table I
R10	Any	Any	Any	Sleep	Any	Closed	IoT-J Table I
R11	Any	Any	Any	Any	Closed	Partial	IoT-J Table I
R12	High	Hot	Any	Work	Any	Closed	extension
R13	Low	Any	Clean	Work	Any	Open	extension
R14	Medium	Hot	Clean	Work	Any	Partial	extension
R15	High	Cold	Clean	Work	Any	Open	extension
R16	Medium	Cold	Clean	Work	Any	Open	extension
R17	High	Warm	Clean	Work	Any	Open	extension
R18	Low	Any	Any	Relax	Any	Closed	extension

Why each extension exists: R12 - the paper has no rule for bright + hot during class, which is exactly the heatwave case; R13 - nothing fires for low daylight during class; R14 - Medium + Hot is uncovered; R15-R17 - R5, R2 and R6 only fire when the previous angle already matches, so from any other angle nothing fires (a dead-lock); R18 - evening darkness → close for privacy. With the extensions a full day has 0 hold events in Normal and Heatwave, and 22 hold minutes in Pollution (16:39-17:00, Low light + Poor air), which the hold logic absorbs. Behaviour to expect from the paper's R11 (previous Closed → Partial): it always pulls a closed curtain back towards Partial, so in a heatwave the equilibrium is ≈ 62° (coverage 0.94, $\tau = 0.11$) rather than a hard 75°. This is faithful to the paper; explain it rather than removing R11.

6.5 Worked example A - Light FIS (hand-checkable)

Inputs L = 40 %, h = 11:00, T = 27 °C (classroom slots).

- Light: Low = (45 − 40)/20 = 0.25; Medium = (40 − 35)/20 = 0.25; High = 0.
- Slot: Work = 1 ($9 \le 11 \le 17$); Sleep = Relax = 0.
- Temp: Cold = 0; Warm = (30 − 27)/7.5 = 0.40; Hot = (27 − 25)/5 = 0.40.
- Firing rules: Low·Work·Warm → Medium min(0.25, 1, 0.4) = 0.25; Low·Work·Hot → Medium 0.25; Medium·Work·Warm → High 0.25; Medium·Work·Hot → Medium 0.25.
- Activations: Medium 0.25, High 0.25. Union of the two clipped triangles is symmetric about 65 → **y* = 65.0 %**. Self-test vector `LightFisClass_40_11_27` = 64.99999999999999.

6.6 Worked example B - Curtain FIS

Inputs L = 100 %, T = 35 °C, AQI = 105, h = 12:00, previous θ = 0° (start of a heatwave noon).

- Light High = 1; Temp Hot = 1 (35 ≥ 34), Warm = 0; AQI Clean = (200 − 105)/100 = 0.95, Poor = (105 − 100)/100 = 0.05; Slot Work = 1; prev Open = 1.
- R12 (High, Hot, Work) → Closed 1.0. R8 (High, Poor) → Partial 0.05. R5/R6/R15/R17 need Cold or Warm → 0.
- Centroid of Closed (full) ∪ Partial (clipped at 0.05) = **72.89°** (vector `CurtainFis_100_35_105_12_0` = 72.89285714285715). The small Poor-air membership pulls the blinds 2.1° off fully closed - a nice viva example of multi-sensor blending.

6.7 Replication of CCIS 2024 Table II (paper time slots)

L %	Slot h	T °C	Paper output	This engine	Same class?
4	19	30	28.29 (Low)	27.73 (Low)	yes
9	11	18	69.24 (High)	60.00 (Medium)	no - borderline
20	12	22	55.01 (Medium)	50.00 (Medium)	yes
30	19	34	28.28 (Low)	27.73 (Low)	yes
45	4	23	22.02 (Low)	20.00 (Low)	yes
65	10	25	56.32 (Medium)	53.57 (Medium)	yes
87	14	30	19.44 (Low)	20.00 (Low)	yes

6 of 7 land in the paper's linguistic class; mean absolute difference 2.96 points. The paper does not publish its exact membership parameters or defuzzification resolution, so an exact match is not possible; the class agreement is the meaningful check.

7. Scenario definitions (exact)

Key	Scenario	Blinds θ	LED level La[r] per row
0	Baseline	0° fixed	1 always (24 h)
1	Manager timer	0° fixed	1 if 09:00 ≤ h < 18:00, else 0
2	PIR occupancy	0° fixed	1 if PIR active (occupied or ≤ 10 min since), else 0
3	Uniform dimming (500 lux)	0° fixed	34/65 = 0.52308 always (24 h)
4	Static daylight harvesting	0° fixed	Window row 0 for 09:00 ≤ h < 13:00, else 1; other rows 1 (24 h)
5	Fuzzy lighting (CCIS 2024)	0° fixed	If PIR: Light FIS(LDR_row[r], h, T_in)/100 per row, else 0
6	Fuzzy win-curtain co-regulation (IoT-J 2026)	If PIR: Curtain FIS(LDR_curtain, T_in, AQI, h, θ_prev) (hold if NaN); else 75°	As S5, but daylight seen through the fuzzy-controlled blinds

S0-S4 are the mid-sem strategies redefined so each isolates one idea (the old code mixed PIR into S3/S4). S5 and S6 include PIR gating because both papers' controllers act only on occupancy; therefore the fair comparison for the fuzzy work is **against S2**, not S0.

8. One control step - exact order

```
1 h = k/60; S = Sun(h); T_out = Tout(h); occ = IsOccupied(h); if occ: k_last = k
   PIR = occ or (k − k_last ≤ 10); E_diff = E_max·S; LDR_curtain = min(100, 0.05·E_diff/10)
2 if S6: θ = PIR ? (CurtainFIS(LDR_curtain, T_in, AQI, h, θ) or hold θ if NaN) : 75
   τ = Tau(θ)
3 for r in 0..2: E_day[r] = DF[r]·E_diff·τ; La[r] = scenario rule (Sec. 7)
4 P = Σ 260·La[r]; E_light += P·Δt; E_base += 780·Δt; if occ: E_cool += (0.6·16.8·I_peak·S·τ +
P)/3·Δt
5 E_task[r] = E_day[r] + 955.882·La[r]; if occ: update T_peak, degree-hours, comfort, U
   curtain travel += |θ − θ_prev|; reversal if direction of a change > 0.5° flips
6 T_in += (T_out + 5·S·τ − T_in)·Δt/1; AQI += (AQI_out + 60·occ − AQI)·Δt/0.5
7 k += 1; wrap at 1440 (k_last −= 1440)
```

Decisions in steps 2-3 use the temperature and AQI from the end of the previous step - a one-minute sensor delay, as a real LM35/MQ-135 loop would have.

9. Unity runtime layer

9.1 Stepping

```
acc += Time.deltaTime · minutesPerRealSecond; n = floor(acc); if n > 240: n = 240, acc = 0 else acc −= n; repeat n: sim.Step()
```

Start(): pixelLightCount = max(current, 16); find hardware by name; sim.ResetToMinute(510) - simulates 00:00-08:30 silently so T_{in} and AQI are warm, then zeroes the KPIs. The demo opens at 08:30 with the room empty and blinds closed (S6).

9.2 Visual mapping (cosmetic, never affects numbers)

Element	Mapping
Slats (30)	visual = MoveTowards(visual, sim.ThetaDeg, 40°/s · deltaTime); slat.localRotation = Euler(visual, 0, 0)
Spotlights (12)	enabled = La > 0.001; intensity = 1.0 · La[row]
LED panels (12)	MaterialPropertyBlock _EmissionColor = white · 2 · La[row]
Sun	h = (MinuteOfDay + acc)/60; s = Sun(h); disabled if s = 0; elevation = 5° + 55°·s; azimuth = −60° + 120°·(h − 6)/12; rotation = Euler(elevation, azimuth, 0) → forward Z = cos(az)·cos(el) > 0, so light always travels through the window wall; intensity = 1.2 · s · E_max/20 000

9.3 Keys

Key	Action
0 … 6 (or keypad)	Switch scenario; KPI counters reset; non-S6 scenarios snap blinds to 0°
T	Toggle Normal ↔ Heatwave (state continues: T_in moves with τ = 1 h ≈ 4 s real)
Q	Toggle Normal ↔ Pollution (AQI 250 outdoors → rules R8/R9 → Partial)
B	Run the 24-h batch for 3 weathers × 7 scenarios, show the overlay, write TwinResults.csv to Application.persistentDataPath
[/]	Halve / double speed (1-120 sim-min per second)
Space · R · H	Pause · reset to 08:30 · hide HUD

9.4 HUD content

- Time, speed, pause flag; scenario name; weather; room state (Occupied / Empty (PIR hold) / Empty).
- Sensors: exterior diffuse lux, window LDR %, T_{out}, T_{in}, AQI with Indian NAQI band (Good ≤ 50, Satisfactory ≤ 100, Moderate ≤ 200, Poor ≤ 300, Very Poor ≤ 400, Severe).
- Actuators: model angle, label, visual servo angle, τ, HOLD flag; for each row LED %, daylight lux, task lux.
- Energy: lighting power vs 780 W, F_c now, kWh since last switch vs baseline, cooling estimate, saving %.
- Key hints. Batch overlay on the right when B is on.

10. Expected results - acceptance criteria

Deterministic 24-h runs, 00:00-24:00, 1-minute steps, from a fresh state. Produced by the C# core (Mono) and identical to the Python reference. Your Unity batch must print these numbers to the shown precision; any difference means the implementation deviates from the spec.

Table 10.1 - Normal day

Scen.	Lighting kWh	Save vs S0	Save vs S2	Cooling kWh	Total kWh	Total vs S2	In band %	F_c	T_peak °C	°C·h > 28	Reversals
S0	18.7200	0.00 %	-188.00 %	9.6124	28.3324	-75.84 %	100.0	1.000	32.06	17.37	0
S1	7.0200	62.50 %	-8.00 %	9.6124	16.6324	-3.23 %	100.0	1.000	32.06	17.37	0
S2	6.5000	65.28 %	0.00 %	9.6124	16.1124	0.00 %	100.0	1.000	32.06	17.37	0
S3	9.7920	47.69 %	-50.65 %	8.6204	18.4124	-14.27 %	100.0	1.000	32.06	17.37	0
S4	17.6800	5.56 %	-172.00 %	9.2657	26.9457	-67.24 %	100.0	1.000	32.06	17.37	0
S5	2.7749	85.18 %	57.31 %	8.4253	11.2001	30.49 %	95.0	0.989	32.06	17.37	0
S6	3.0686	83.61 %	52.79 %	5.9541	9.0227	44.00 %	94.4	0.985	29.50	6.61	5

Table 10.2 - Heatwave day

Scen.	Lighting kWh	Save vs S0	Save vs S2	Cooling kWh	Total kWh	Total vs S2	In band %	F_c	T_peak °C	°C·h > 28	Reversals
S0	18.7200	0.00 %	-188.00 %	12.4371	31.1571	-64.53 %	56.9	0.979	42.06	95.79	0
S1	7.0200	62.50 %	-8.00 %	12.4371	19.4571	-2.75 %	56.9	0.979	42.06	95.79	0
S2	6.5000	65.28 %	0.00 %	12.4371	18.9371	0.00 %	56.9	0.979	42.06	95.79	0
S3	9.7920	47.69 %	-50.65 %	11.4451	21.2371	-12.15 %	100.0	1.000	42.06	95.79	0
S4	17.6800	5.56 %	-172.00 %	12.0904	29.7704	-57.21 %	97.1	1.000	42.06	95.79	0
S5	2.6980	85.59 %	58.49 %	11.2243	13.9223	26.48 %	95.2	0.990	42.06	95.79	0
S6	3.1655	83.09 %	51.30 %	2.5325	5.6980	69.91 %	94.6	0.986	38.43	70.23	7

Table 10.3 - Pollution day

Scen.	Lighting kWh	Save vs S0	Save vs S2	Cooling kWh	Total kWh	Total vs S2	In band %	F_c	T_peak °C	°C·h > 28	Reversals
S0	18.7200	0.00 %	-188.00 %	9.6124	28.3324	-75.84 %	100.0	1.000	32.06	17.37	0
S1	7.0200	62.50 %	-8.00 %	9.6124	16.6324	-3.23 %	100.0	1.000	32.06	17.37	0
S2	6.5000	65.28 %	0.00 %	9.6124	16.1124	0.00 %	100.0	1.000	32.06	17.37	0
S3	9.7920	47.69 %	-50.65 %	8.6204	18.4124	-14.27 %	100.0	1.000	32.06	17.37	0
S4	17.6800	5.56 %	-172.00 %	9.2657	26.9457	-67.24 %	100.0	1.000	32.06	17.37	0
S5	2.7749	85.18 %	57.31 %	8.4253	11.2001	30.49 %	95.0	0.989	32.06	17.37	0
S6	3.1623	83.11 %	51.35 %	3.0873	6.2496	61.21 %	93.1	0.978	28.69	1.74	17

10.1 What the numbers say (use these in the report)

- **Fuzzy lighting is the big lighting win.** S5 uses 2.7749 kWh against 6.5000 kWh for PIR-only S2: 57.31 % less lighting energy (Normal), from per-row daylight-linked dimming.
- **Curtains trade a little light for a lot of heat.** On a normal day S6 uses +10.6 % lighting energy vs S5 (partially closed blinds admit less daylight) but -29.3 % solar cooling load, for -19.4 % total controllable energy, and lowers the indoor peak from 32.06 °C to 29.50 °C.
- **The heatwave is where S6 earns its place.** Lighting +17.3 %, cooling -77.4 %, total -59.1 % vs S5; peak 42.06 → 38.43 °C; degree-hours above 28 °C 95.79 → 70.23.
- **Comparing with the paper.** IoT-J reports 18 % (light only) and 39.45 % (light + curtain) reduction of artificial light (Eq. 12). Here Eq. 12 gives 57.07 % (S5) and 52.60 % (S6) on a normal day. The direction differs because the paper's baseline window is shaded while this twin's legacy window is bare - state this explicitly in the viva rather than claiming the paper's number.
- Headline vs the 24/7 baseline: S6 saves 83.61 % lighting energy, but 65.28 percentage points of that is just switching off when nobody is there (S2 = 65.28 %). Quote both.
- Mid-sem S0-S4 numbers change slightly from the old slides (S2 65.28 %% instead of 63-64 %%, S3 47.69 %% instead of 46 %%, S4 5.56 %% instead of 8 %%) because they are now computed from stated definitions.

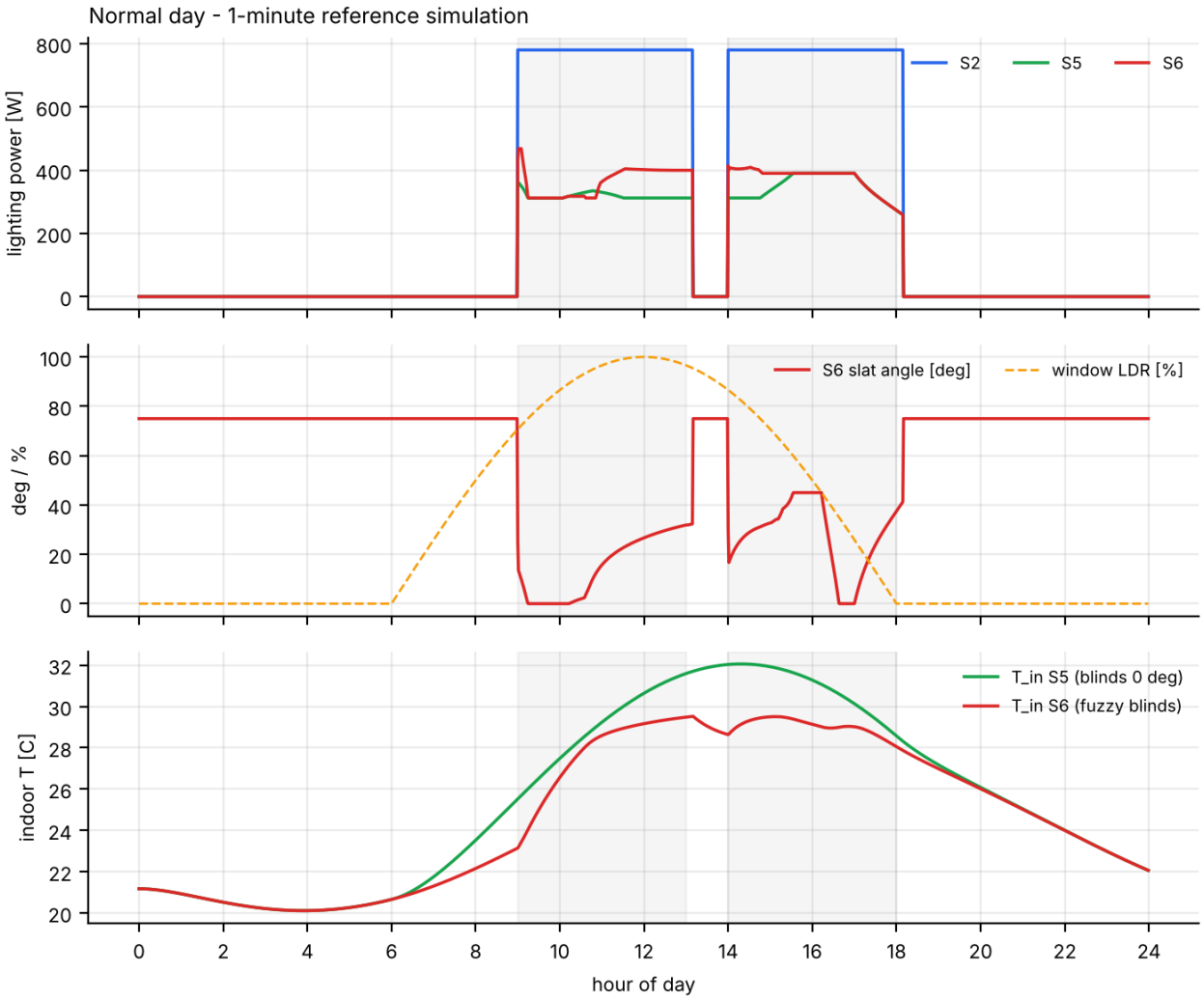


Figure 10.1 - Normal day. Blinds close when the room empties (lunch, evening), open to 0° in the bright morning, drift to ~30° as T_{in} approaches 28-30 °C, go Partial when daylight is Medium (rule R3), and reopen when daylight is Low (R13).

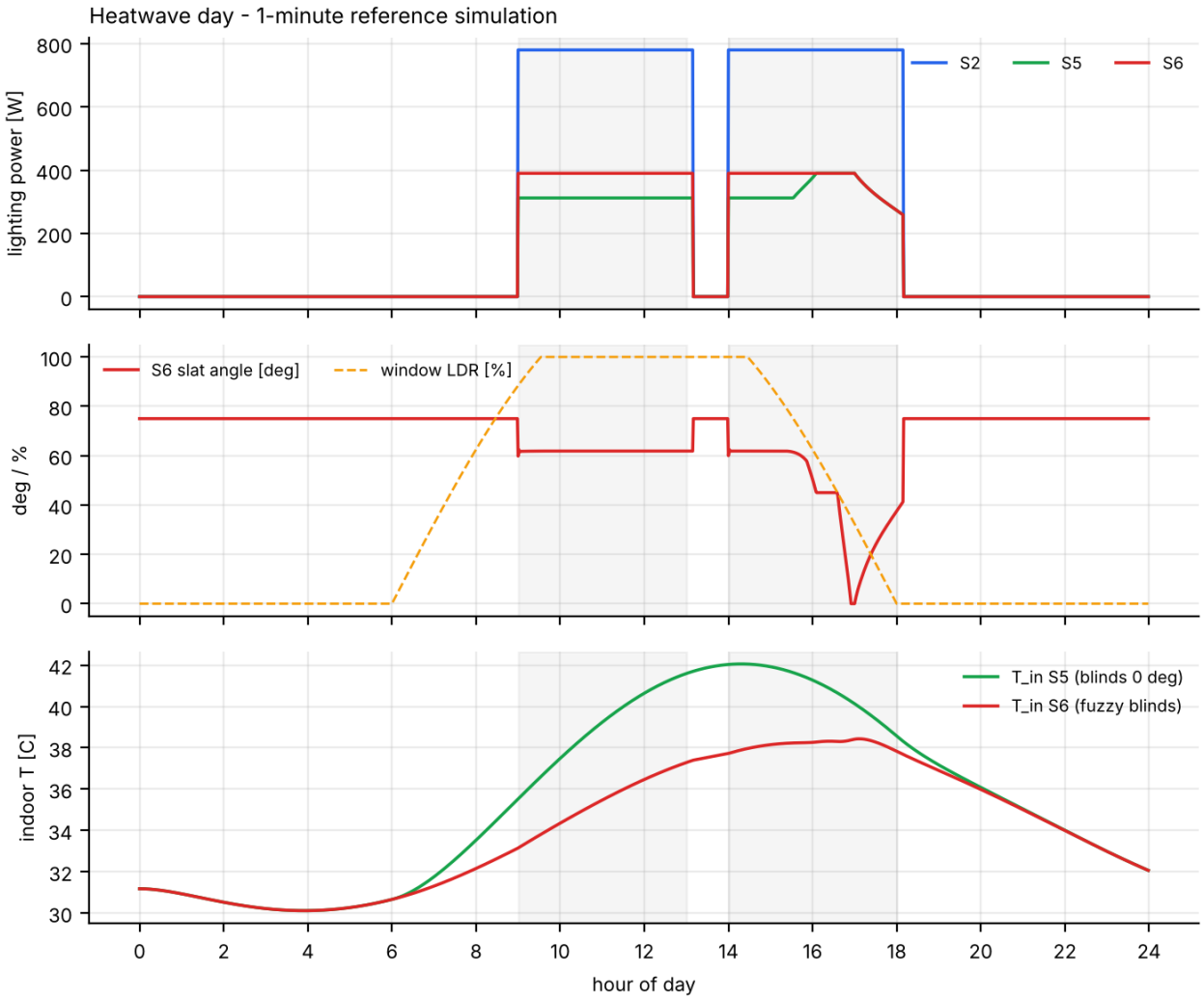


Figure 10.2 - Heatwave day. Rule R12 holds the blinds at ≈ 60-62° (94 % coverage) through the teaching day, opening only for low evening daylight; LEDs run at ≈ 50 %; the indoor peak stays 3.63 °C below S5.

11. Build order and verification

Step	Action	Check before continuing
1	Back up the project (copy the folder or git commit).	-
2	Project settings: Edit → Project Settings → Player → Active Input Handling = Input Manager (Old) or Both. Graphics → Built-in RP (no Scriptable Render Pipeline asset). Optional: Player → Color Space Linear.	Editor restarts if input handling changed.
3	Copy the 5 Core files into Assets/Scripts/Core/ (new folder).	Console: 0 errors. These files compile on their own (no UnityEngine).
4	Replace Assets/Scripts/SimulationEngine.cs, Assets/Editor/ClassroomBuilder.cs and Assets/Editor/SimulationBuilder.cs.	0 compile errors; menu 'Digital Twin' shows 4 items.
5	Digital Twin → Run Self-Test.	Exactly 'SELF-TEST PASSED (98 checks)'. Any FAIL: stop and diff the Core files against the zip.
6	Digital Twin → Generate Classroom (Paper Figure 5 Type B).	Smart_Blinds has 3 bays × 10 slats; Window_Glass has 3 panes; Ceiling exists; Assets/Materials/Twin has 9 .mat files.
7	Digital Twin → Add Simulation Engine; save the scene.	Simulation_Engine has the component; Main Camera at (5.0, 2.2, 3.9) looking at the window wall.
8	Digital Twin → Run 24h Batch Evaluation.	TwinResults/TwinResults.csv (next to Assets/) matches Tables 10.1-10.3.

Step	Action	Check before continuing
9	Press Play.	08:30, room Empty, blinds 75° (S6). At 09:00 Occupied; blinds swing open within ~2 s real; window-row LED ≈ 66 % then 20 % by 09:30; HUD refreshes ~15 sim-minutes per second.
10	At about 11:00 press T.	Window-row LED rises from ≈ 39 % to 50 %; T_{in} climbs (28.6 → 32.2 °C in 30 sim-min); blinds close to ≈ 61-62° within ~2 real seconds and stay there (R11 equilibrium, see 6.4).
11	Press B.	Overlay equals Table 10; CSV path printed in the Console.

11.1 Troubleshooting

Symptom	Cause → fix
Everything is pink	Project uses URP/HDRP. Builder supports URP Lit materials; for HDRP switch to Built-in or adapt only the Mat() helper.
InvalidOperationException from Input	New Input System only → set Active Input Handling to Both.
Only a few LED fixtures light the floor	pixelLightCount too low or Deferred off → check QualitySettings; Spotlights use ForcePixel.
No slat shadows on desks	Sun disabled at night (06:00-18:00 only), or Shadow Distance < 15 m in Quality settings, or glass casting shadows (must be Off).
'Smart_Blinds not found' error	Generate Classroom was not run in this scene, or the scene was not saved after generating.
Batch differs in the 4th decimal	A constant, rule or the step order was changed; diff the Core files against the zip.
Self-test fails only on Day_* checks	Step order or accumulation changed (e.g. float used instead of double).

12. Viva demonstration script and honest limitations

12.1 Script (about 4 minutes)

- Press 2 (PIR): lights full on whenever occupied - this is the mid-sem state of the art.
- Press 5: the three rows dim independently; point at Window ≈ 20 % vs Corridor ≈ 50 %. Explain the 27 CCIS rules and the centroid (worked example A).
- Press 6: blinds track the day; read the HUD angle and τ. Explain co-regulation - the light FIS now sees daylight through the blinds.
- Press T: heatwave. Rule R12 fires; blinds close to ≈ 62° (R11 keeps them off a hard 75°); the window-row LED rises to 50 %. Worked example B shows how a little poor air shifts 75° to 72.9°.
- Press Q: pollution - Poor-air rules R8/R9 force Partial.
- Press B: the deterministic 24-h table. Lead with 'vs S2' and the heatwave total-energy and temperature results; show the self-test passing.

12.2 Limitations to state before the examiner does

- Daylight-factor model, no ray-traced daylight; the Unity sun and shadows are visual and do not feed the numbers.
- Indoor temperature is a first-order model of a fan-ventilated room; the cooling figure is an estimate of what an AC would need, not a measured value.
- Membership parameters and the classroom time slots are engineering choices; the papers do not publish exact values.
- The papers' rule bases are used verbatim, including their non-monotonic light rules; seven extension rules were added only to fill coverage gaps.
- One synthetic day per weather profile; real validation would replay measured sensor logs (the CSV format makes that a small extension).

Appendix A - Complete source code

Authoritative copies are in **ClassroomTwin_Source.zip**. In this listing a line that was too long for the page continues on the next line after the marker »»; the marker is not part of the code. All 8 files were compiled together with Mono against UnityEngine/UnityEditor API stubs (0 errors, 0 warnings), and the 5 Core files were compiled and executed for real.

Assets/Scripts/Core/FuzzyCore.cs

```
// ClassroomTwin - Mamdani fuzzy inference core (pure C#, no UnityEngine dependency).
// Implements: trapezoidal/triangular membership functions with shoulders, min (AND),
// max (aggregation), clipped (min) implication, discrete centroid defuzzification.
using System;
using System.Collections.Generic;

namespace ClassroomTwin.Core
{
    /// <summary>Trapezoid (a,b,c,d). Triangle = (a,b,b,c). a==b => left shoulder, c==d => right shoulder.</summary>
    public sealed class FuzzySet
    {
        public readonly string Name;
        public readonly double A, B, C, D;

        public FuzzySet(string name, double a, double b, double c, double d)
        {
            if (!(a <= b && b <= c && c <= d)) throw new ArgumentException("FuzzySet " + name + ": need a<=b<=c<=d");
            Name = name; A = a; B = b; C = c; D = d;
        }

        public static FuzzySet Tri(string name, double a, double b, double c) { return new FuzzySet(name, a, b, b, c); }
        public static FuzzySet Trap(string name, double a, double b, double c, double d) { return new FuzzySet(name, a, b, c,
»» d); }

        public double Mu(double x)
        {
            if (x < A || x > D) return 0.0;
            if (x >= B && x <= C) return 1.0;
            if (x < B) return (B == A) ? 1.0 : (x - A) / (B - A);
            return (D == C) ? 1.0 : (D - x) / (D - C);
        }
    }

    /// <summary>A linguistic variable. Crisp inputs are clamped to [Min, Max] before fuzzification.</summary>
    public sealed class FuzzyVariable
    {
        public readonly string Name;
        public readonly double Min, Max;
        public readonly List<FuzzySet> Sets = new List<FuzzySet>();

        public FuzzyVariable(string name, double min, double max) { Name = name; Min = min; Max = max; }

        public FuzzyVariable Add(FuzzySet s) { Sets.Add(s); return this; }

        public int IndexOf(string setName)
        {
            for (int i = 0; i < Sets.Count; i++) if (Sets[i].Name == setName) return i;
            throw new ArgumentException("Variable " + Name + " has no set " + setName);
        }

        public double Clamp(double x) { return x < Min ? Min : (x > Max ? Max : x); }
    }

    /// <summary>IF in0 is S0 AND in1 is S1 ... THEN out is O. Antecedent index -1 means "Any" (membership 1).</summary>
    public sealed class FuzzyRule
    {
        public readonly string Id;
        public readonly int[] Antecedents;
        public readonly int Consequent;
        public FuzzyRule(string id, int[] antecedents, int consequent) { Id = id; Antecedents = antecedents; Consequent =
»» consequent; }
    }

    public sealed class MamdaniFIS
    {
        public readonly string Name;
        public readonly List<FuzzyVariable> Inputs = new List<FuzzyVariable>();
        public readonly FuzzyVariable Output;
        public readonly double UMin, UMax, UStep;     // discretised output universe
        public readonly double ClampMin, ClampMax;    // final crisp output clamp
        public readonly List<FuzzyRule> Rules = new List<FuzzyRule>();

        public MamdaniFIS(string name, FuzzyVariable output, double uMin, double uMax, double uStep, double clampMin, double
»» clampMax)
        {
            Name = name; Output = output; UMin = uMin; UMax = uMax; UStep = uStep; ClampMin = clampMin; ClampMax = clampMax;
        }

        public MamdaniFIS AddInput(FuzzyVariable v) { Inputs.Add(v); return this; }

        /// <summary>Add a rule using set names; null or "Any" = don't care.</summary>
        public MamdaniFIS AddRule(string id, string[] antecedentNames, string consequentName)
        {
            if (antecedentNames.Length != Inputs.Count) throw new ArgumentException("Rule " + id + ": wrong antecedent count");
```

```
            int[] idx = new int[antecedentNames.Length];
            for (int i = 0; i < idx.Length; i++)
                idx[i] = (antecedentNames[i] == null || antecedentNames[i] == "Any") ? -1 : Inputs[i].IndexOf(antecedentNames[
»» i]);
            Rules.Add(new FuzzyRule(id, idx, Output.IndexOf(consequentName)));
            return this;
        }

        /// <summary>
        /// Evaluates the FIS. Returns the clamped centroid, or double.NaN when no rule fires
        /// (caller must hold its previous output). activations (length = Output.Sets.Count) receives
        /// the max rule strength per output set; may be null.
        /// </summary>
        public double Evaluate(double[] crisp, double[] activations)
        {
            int nIn = Inputs.Count;
            double[][] mu = new double[nIn][];
            for (int i = 0; i < nIn; i++)
            {
                FuzzyVariable v = Inputs[i];
                double x = v.Clamp(crisp[i]);
                mu[i] = new double[v.Sets.Count];
                for (int j = 0; j < v.Sets.Count; j++) mu[i][j] = v.Sets[j].Mu(x);
            }

            int nOut = Output.Sets.Count;
            double[] act = activations ?? new double[nOut];
            for (int o = 0; o < nOut; o++) act[o] = 0.0;

            for (int r = 0; r < Rules.Count; r++)
            {
                FuzzyRule rule = Rules[r];
                double a = 1.0;
                for (int i = 0; i < nIn; i++)
                {
                    int s = rule.Antecedents[i];
                    double m = (s < 0) ? 1.0 : mu[i][s];
                    if (m < a) a = m;
                }
                if (a > act[rule.Consequent]) act[rule.Consequent] = a;
            }

            int n = (int)Math.Round((UMax - UMin) / UStep) + 1;
            double num = 0.0, den = 0.0;
            for (int k = 0; k < n; k++)
            {
                double y = UMin + k * UStep;
                double m = 0.0;
                for (int o = 0; o < nOut; o++)
                {
                    if (act[o] <= 0.0) continue;
                    double v = Output.Sets[o].Mu(y);
                    if (act[o] < v) v = act[o];
                    if (v > m) m = v;
                }
                num += y * m;
                den += m;
            }
            if (den <= 1e-12) return double.NaN;
            double c = num / den;
            return c < ClampMin ? ClampMin : (c > ClampMax ? ClampMax : c);
        }
    }
}
```

Assets/Scripts/Core/TwinConfig.cs

```
// ClassroomTwin - every physical constant and assumption in one place (pure C#).
// Changing a value here changes the batch results; the reference numbers in the spec
// were produced with exactly these values.
using System;

namespace ClassroomTwin.Core
{
    public enum Weather { Normal = 0, Heatwave = 1, Pollution = 2 }

    public sealed class WeatherProfile
    {
        public readonly string Name;
        public readonly double EdiffMaxLux;   // peak exterior diffuse horizontal illuminance
        public readonly double TMeanC;        // daily mean outdoor temperature
        public readonly double TAmpC;         // half peak-to-peak diurnal swing, peak at 15:00
        public readonly double DTGainC;       // max indoor solar temperature rise with blinds fully open
        public readonly double AqiOut;        // outdoor AQI
        public readonly double IFacadePeak;   // peak solar irradiance on the window facade [W/m2]

        public WeatherProfile(string name, double ediff, double tmean, double tamp, double dtgain, double aqiOut, double iPeak)
        {
            Name = name; EdiffMaxLux = ediff; TMeanC = tmean; TAmpC = tamp; DTGainC = dtgain; AqiOut = aqiOut; IFacadePeak =
»» iPeak;
        }
    }

    public static class TwinConfig
    {
        // ---------- Simulation timing ----------
        public const int StepMinutes = 1;                    // fixed control/physics step (simulated minutes)
```

```
        public const int MinutesPerDay = 1440;
        public const double StepHours = StepMinutes / 60.0;

        // ---------- Lighting plant (MDPI 2022 mid-sem model) ----------
        public const int Rows = 3;                          // 0 = Window, 1 = Center, 2 = Corridor
        public const int FixturesPerRow = 4;
        public const int TotalFixtures = Rows * FixturesPerRow; // 12
        public const double FixtureFullW = 65.0;            // W at 100 %
        public const double FixtureDimW = 34.0;             // W that yields 500 lux (MDPI dimming case)
        public const double TargetLux = 500.0;
        public const double ArtificialLuxFull = TargetLux * FixtureFullW / FixtureDimW; // 955.882352941... lux per zone at
»» 100 %
        public const double BaselinePowerW = TotalFixtures * FixtureFullW;              // 780 W
        public static readonly string[] RowNames = { "Window", "Center", "Corridor" };

        // ---------- Daylight (daylight-factor method) ----------
        public static readonly double[] DaylightFactor = { 0.050, 0.020, 0.008 }; // per row, blinds open
        public const double LdrFullScaleLux = 1000.0;       // lux that reads as 100 % on the LDR scale

        // ---------- Venetian blinds ----------
        public const double SlatDepthM = 0.17;
        public const double SlatPitchM = 0.16;
        public const double SlatTransmittance = 0.05;
        public const double StaticBlindAngleDeg = 0.0;      // S0-S5: legacy unshaded window (slats horizontal)
        public const double ClosedAngleDeg = 75.0;          // IoT-J Fig. 3 fully closed
        public const double PartialAngleDeg = 45.0;
        public const double OpenAngleDeg = 0.0;

        // ---------- Occupancy / PIR ----------
        public const int PirHoldMinutes = 10;               // lights stay on 10 min after last motion

        // ---------- Thermal / air ----------
        public const double ThermalTauH = 1.0;              // indoor temperature time constant
        public const double AirTauH = 0.5;                  // indoor AQI time constant
        public const double AqiOccupancyGain = 60.0;        // AQI rise caused by a full class
        public const double DegreeHourBaseC = 28.0;

        // ---------- Cooling-load estimate (KPI only, does not feed back) ----------
        public const double WindowAreaM2 = 3 * 3.5 * 1.6;   // 16.8 m2
        public const double Shgc = 0.60;
        public const double AcCop = 3.0;

        // ---------- Comfort (useful illuminance band) ----------
        public const double ComfortLowLux = 400.0;
        public const double ComfortHighLux = 2000.0;
        public static readonly FuzzySet ComfortMembership = FuzzySet.Trap("Comfort", 300.0, 400.0, 2000.0, 3000.0);

        // ---------- Weather profiles ----------
        public static readonly WeatherProfile[] Profiles =
        {
            new WeatherProfile("Normal",    20000.0, 24.0, 4.0, 5.0,  45.0, 400.0),
            new WeatherProfile("Heatwave",  25000.0, 34.0, 4.0, 5.0,  45.0, 550.0),
            new WeatherProfile("Pollution", 20000.0, 24.0, 4.0, 5.0, 250.0, 400.0),
        };

        public static WeatherProfile Profile(Weather w) { return Profiles[(int)w]; }

        public static readonly string[] ScenarioNames =
        {
            "S0 Baseline (24/7 ON)",
            "S1 Manager (ON 09:00-18:00)",
            "S2 PIR occupancy (10-min hold)",
            "S3 Uniform dimming 34 W (24/7)",
            "S4 Static daylight (window row OFF 09-13)",
            "S5 Fuzzy lighting - IEEE CCIS 2024",
            "S6 Fuzzy win-curtain co-regulation - IEEE IoT-J 2026",
        };
    }
}
```

Assets/Scripts/Core/TwinControllers.cs

```
// ClassroomTwin - the two Mamdani controllers.
//  * Light FIS   : IEEE CCIS 2024 (Singhai et al.), Table I, 27 rules, 3 inputs.
//  * Curtain FIS : IEEE IoT-J 2026 (Singhai et al.), Table I rules R1-R11 + documented extensions R12-R18.
using System;

namespace ClassroomTwin.Core
{
    public static class TwinControllers
    {
        // ---------------- shared input variables ----------------
        public static FuzzyVariable LightInput()
        {
            return new FuzzyVariable("Light", 0, 100)
                .Add(FuzzySet.Trap("Low", 0, 0, 25, 45))
                .Add(FuzzySet.Tri("Medium", 35, 55, 75))
                .Add(FuzzySet.Trap("High", 60, 80, 100, 100));
        }

        /// <summary>Classroom time slots (adapted from the paper's home slots). paperSlots=true uses Fig. 6(b) of CCIS 2024.</
»» summary>
        public static FuzzyVariable SlotInput(bool paperSlots)
        {
            FuzzyVariable v = new FuzzyVariable("Slot", 0, 24);
            if (paperSlots)
            {
```

```
            v.Add(FuzzySet.Trap("Sleep", 0, 0, 5, 10))
             .Add(FuzzySet.Tri("Work", 8, 14, 20))
             .Add(FuzzySet.Trap("Relax", 18, 20, 24, 24));
        }
        else
        {
            v.Add(FuzzySet.Trap("Sleep", 0, 0, 6, 8))
             .Add(FuzzySet.Trap("Work", 7, 9, 17, 19))
             .Add(FuzzySet.Trap("Relax", 17, 19, 24, 24));
        }
        return v;
    }

    // ---------------- Light FIS (CCIS 2024) ----------------
    public static MamdaniFIS BuildLightFis(bool paperSlots)
    {
        FuzzyVariable temp = new FuzzyVariable("Temp", 0, 40)
            .Add(FuzzySet.Trap("Cold", 0, 0, 10, 20))
            .Add(FuzzySet.Tri("Warm", 15, 22.5, 30))
            .Add(FuzzySet.Trap("Hot", 25, 30, 40, 40));

        FuzzyVariable output = new FuzzyVariable("Artificial", 0, 100)
            .Add(FuzzySet.Tri("Low", -10, 20, 50))
            .Add(FuzzySet.Tri("Medium", 20, 50, 80))
            .Add(FuzzySet.Tri("High", 50, 80, 110));

        MamdaniFIS fis = new MamdaniFIS("LightFIS_CCIS2024", output, -10.0, 110.0, 0.5, 0.0, 100.0);
        fis.AddInput(LightInput()).AddInput(SlotInput(paperSlots)).AddInput(temp);

        // CCIS 2024 Table I.  Order of temperature columns: Cold, Warm, Hot.
        string[] L = { "Low", "Medium", "High" };
        string[] S = { "Work", "Relax", "Sleep" };
        string[] T = { "Cold", "Warm", "Hot" };
        string[,,] table =
        {
            { { "High", "Medium", "Medium" }, { "Medium", "Medium", "Low" }, { "Low", "Low", "Low" } },      // Light Low
            { { "High", "High", "Medium" },   { "Medium", "Medium", "Medium" }, { "Low", "Low", "Low" } },   // Light
»» Medium
            { { "Medium", "Low", "Low" },     { "Medium", "Medium", "Low" }, { "Medium", "Low", "Low" } },   // Light High
        };
        int n = 1;
        for (int l = 0; l < 3; l++)
            for (int s = 0; s < 3; s++)
                for (int t = 0; t < 3; t++)
                    fis.AddRule("L" + (n++), new[] { L[l], S[s], T[t] }, table[l, s, t]);
        return fis;
    }

    // ---------------- Curtain FIS (IoT-J 2026) ----------------
    public static FuzzyVariable AngleVariable()
    {
        return new FuzzyVariable("Angle", 0, 75)
            .Add(FuzzySet.Tri("Open", -30, 0, 30))
            .Add(FuzzySet.Tri("Partial", 15, 45, 75))
            .Add(FuzzySet.Tri("Closed", 45, 75, 105));
    }

    public static MamdaniFIS BuildCurtainFis(bool paperSlots, bool includeExtensions)
    {
        FuzzyVariable temp = new FuzzyVariable("Temp", 0, 40)
            .Add(FuzzySet.Trap("Cold", 0, 0, 16, 22))
            .Add(FuzzySet.Tri("Warm", 18, 25, 32))
            .Add(FuzzySet.Trap("Hot", 28, 34, 40, 40));
        FuzzyVariable aqi = new FuzzyVariable("AQI", 0, 500)
            .Add(FuzzySet.Trap("Clean", 0, 0, 100, 200))
            .Add(FuzzySet.Trap("Poor", 100, 200, 500, 500));

        MamdaniFIS fis = new MamdaniFIS("CurtainFIS_IoTJ2026", AngleVariable(), -30.0, 105.0, 0.5, 0.0, 75.0);
        fis.AddInput(LightInput()).AddInput(temp).AddInput(aqi).AddInput(SlotInput(paperSlots)).AddInput(AngleVariable());

        //            Light     Temp    AQI      Slot     PrevAngle   ->  NewAngle
        fis.AddRule("R1",  new[] { "Low",    null,   null,    "Sleep", null },      "Closed");
        fis.AddRule("R2",  new[] { "Medium", "Cold", "Clean", "Work",  "Open" },    "Open");
        fis.AddRule("R3",  new[] { "Medium", "Warm", "Clean", "Work",  null },      "Partial");
        fis.AddRule("R4",  new[] { "Medium", null,   null,    "Relax", null },      "Partial");
        fis.AddRule("R5",  new[] { "High",   "Cold", "Clean", "Work",  "Open" },    "Open");
        fis.AddRule("R6",  new[] { "High",   "Warm", "Clean", "Work",  "Partial" }, "Open");
        fis.AddRule("R7",  new[] { "High",   "Hot",  "Clean", "Relax", null },      "Partial");
        fis.AddRule("R8",  new[] { "High",   null,   "Poor",  null,    null },      "Partial");
        fis.AddRule("R9",  new[] { "Medium", null,   "Poor",  null,    null },      "Partial");
        fis.AddRule("R10", new[] { null,     null,   null,    "Sleep", null },      "Closed");
        fis.AddRule("R11", new[] { null,     null,   null,    null,    "Closed" },  "Partial");
        if (includeExtensions)
        {
            fis.AddRule("R12", new[] { "High",   "Hot",  null,    "Work",  null },  "Closed");  // thermal / glare
»» protection
            fis.AddRule("R13", new[] { "Low",    null,   "Clean", "Work",  null },  "Open");    // harvest scarce daylight
            fis.AddRule("R14", new[] { "Medium", "Hot",  "Clean", "Work",  null },  "Partial");
            fis.AddRule("R15", new[] { "High",   "Cold", "Clean", "Work",  null },  "Open");    // R5 without prev-angle
»» condition
            fis.AddRule("R16", new[] { "Medium", "Cold", "Clean", "Work",  null },  "Open");    // R2 without prev-angle
»» condition
            fis.AddRule("R17", new[] { "High",   "Warm", "Clean", "Work",  null },  "Open");    // R6 without prev-angle
»» condition
            fis.AddRule("R18", new[] { "Low",    null,   null,    "Relax", null },  "Closed");  // evening privacy
        }
        return fis;
```

```
        }
    }
}
```

Assets/Scripts/Core/TwinSimulator.cs

```
// ClassroomTwin - deterministic fixed-step digital-twin simulator (pure C#).
// One call to Step() advances exactly TwinConfig.StepMinutes simulated minutes.
// The Unity layer (SimulationEngine) and the 24-h batch evaluator both use this class,
// so the live demo and the reported numbers can never disagree.
using System;

namespace ClassroomTwin.Core
{
    public sealed class DayResult
    {
        public int Scenario;
        public Weather Weather;
        public double LightingKWh, CoolingKWh, TotalKWh;
        public int OccupiedMinutes, ComfortMinutes, HeldSteps, Reversals;
        public double ComfortBandPct, FcAvg, UAvgPct, ReducedArtificialPct, TPeakC, DegreeHours28, TravelDeg;
    }

    public sealed class TwinSimulator
    {
        // ---------- configuration ----------
        public int Scenario;
        public Weather Weather;
        public readonly bool PaperSlots;
        readonly MamdaniFIS lightFis;
        readonly MamdaniFIS curtainFis;

        // ---------- state ----------
        public int MinuteOfDay;            // 0..1439, time at the START of the next step
        public double ThetaDeg;            // modelled curtain slat angle (servo output)
        public double TinC;                // indoor temperature (free-running, fan-ventilated room)
        public double Aqi;                 // indoor AQI
        int lastOccupiedMinute = -100000;

        // ---------- last-step telemetry (read by HUD) ----------
        public double HourOfDay, SunFactor, EdiffLux, LdrCurtainPct, PowerW, TauBlind, ToutC;
        public bool Occupied, PirActive, CurtainHeld;
        public readonly double[] La = new double[TwinConfig.Rows];          // 0..1 per row
        public readonly double[] EdayLux = new double[TwinConfig.Rows];
        public readonly double[] EtaskLux = new double[TwinConfig.Rows];
        public readonly double[] LightActs = new double[3];                  // last evaluated row (diagnostics)
        public readonly double[] CurtainActs = new double[3];
        public double FcNow;

        // ---------- KPI accumulators (since last ResetKpis) ----------
        public double LightingWh, CoolingWh, BaselineWh, ComfortSum, UtilSum, TPeak, DegreeHours, Travel;
        public int OccMin, ComfortMin, Held, Reversals;
        double prevTheta; int lastDir;

        public TwinSimulator(int scenario, Weather weather, bool paperSlots = false, bool curtainExtensions = true)
        {
            Scenario = scenario; Weather = weather; PaperSlots = paperSlots;
            lightFis = TwinControllers.BuildLightFis(paperSlots);
            curtainFis = TwinControllers.BuildCurtainFis(paperSlots, curtainExtensions);
            ResetState();
        }

        // ================ environment model (static, unit-testable) ================
        public static double Sun(double h) { return (h >= 6.0 && h <= 18.0) ? Math.Sin(Math.PI * (h - 6.0) / 12.0) : 0.0; }

        public static double Tout(double h, WeatherProfile w) { return w.TMeanC + w.TAmpC * Math.Cos(2.0 * Math.PI * (h - 15.
»» 0) / 24.0); }

        public static double Coverage(double thetaDeg)
        {
            double c = TwinConfig.SlatDepthM * Math.Sin(thetaDeg * Math.PI / 180.0) / TwinConfig.SlatPitchM;
            return c > 1.0 ? 1.0 : c;
        }

        public static double Tau(double thetaDeg) { return 1.0 - Coverage(thetaDeg) * (1.0 - TwinConfig.SlatTransmittance); }

        public static bool IsOccupied(double h) { return (h >= 9.0 && h < 13.0) || (h >= 14.0 && h < 18.0); }

        // ================ lifecycle ================
        public void ResetState()
        {
            WeatherProfile w = TwinConfig.Profile(Weather);
            MinuteOfDay = 0;
            ThetaDeg = (Scenario == 6) ? TwinConfig.ClosedAngleDeg : TwinConfig.StaticBlindAngleDeg;
            TinC = Tout(0.0, w);
            Aqi = w.AqiOut;
            lastOccupiedMinute = -100000;
            ResetKpis();
        }

        public void ResetKpis()
        {
            LightingWh = CoolingWh = BaselineWh = ComfortSum = UtilSum = DegreeHours = Travel = 0.0;
            TPeak = double.NegativeInfinity;
            OccMin = ComfortMin = Held = Reversals = 0;
            prevTheta = ThetaDeg; lastDir = 0;
        }
```

```
        /// <summary>Switch scenario while running (live demo). Non-curtain scenarios snap blinds to the static angle.</
»» summary>
        public void SetScenario(int s)
        {
            Scenario = s;
            if (s != 6) ThetaDeg = TwinConfig.StaticBlindAngleDeg;
            ResetKpis();
        }

        /// <summary>Fresh day, silently simulated from 00:00 up to startMinute so physical state is warm.</summary>
        public void ResetToMinute(int startMinute)
        {
            ResetState();
            for (int i = 0; i < startMinute; i++) Step();
            ResetKpis();
        }

        // ================= one fixed step =================
        public void Step()
        {
            WeatherProfile w = TwinConfig.Profile(Weather);
            double dt = TwinConfig.StepHours;
            int k = MinuteOfDay;
            double h = k / 60.0;
            HourOfDay = h;

            // 1. exogenous inputs
            SunFactor = Sun(h);
            ToutC = Tout(h, w);
            Occupied = IsOccupied(h);
            if (Occupied) lastOccupiedMinute = k;
            PirActive = Occupied || (k - lastOccupiedMinute) <= TwinConfig.PirHoldMinutes;
            EdiffLux = w.EdiffMaxLux * SunFactor;
            LdrCurtainPct = Math.Min(100.0, TwinConfig.DaylightFactor[0] * EdiffLux / TwinConfig.LdrFullScaleLux * 100.0);

            // 2. curtain decision (S6 only)
            CurtainHeld = false;
            if (Scenario == 6)
            {
                if (PirActive)
                {
                    double y = curtainFis.Evaluate(new[] { LdrCurtainPct, TinC, Aqi, h, ThetaDeg }, CurtainActs);
                    if (double.IsNaN(y)) { CurtainHeld = true; Held++; }   // no rule fired: hold position
                    else ThetaDeg = y;
                }
                else ThetaDeg = TwinConfig.ClosedAngleDeg;                  // room empty: close (heat gain / privacy)
            }
            TauBlind = Tau(ThetaDeg);

            // 3. lighting decision
            for (int r = 0; r < TwinConfig.Rows; r++)
            {
                EdayLux[r] = TwinConfig.DaylightFactor[r] * EdiffLux * TauBlind;
                La[r] = LightingLevel(r, h);
            }

            // 4. energy
            PowerW = 0.0;
            for (int r = 0; r < TwinConfig.Rows; r++) PowerW += TwinConfig.FixturesPerRow * TwinConfig.FixtureFullW * La[r];
            LightingWh += PowerW * dt;
            BaselineWh += TwinConfig.BaselinePowerW * dt;
            if (Occupied)
            {
                double qSolar = TwinConfig.Shgc * TwinConfig.WindowAreaM2 * w.IFacadePeak * SunFactor * TauBlind;
                CoolingWh += (qSolar + PowerW) / TwinConfig.AcCop * dt;
            }

            // 5. comfort + KPIs
            double fcSum = 0.0; bool allInBand = true; double laSum = 0.0;
            for (int r = 0; r < TwinConfig.Rows; r++)
            {
                EtaskLux[r] = EdayLux[r] + TwinConfig.ArtificialLuxFull * La[r];
                fcSum += TwinConfig.ComfortMembership.Mu(EtaskLux[r]);
                if (EtaskLux[r] < TwinConfig.ComfortLowLux || EtaskLux[r] > TwinConfig.ComfortHighLux) allInBand = false;
                laSum += La[r];
            }
            FcNow = fcSum / TwinConfig.Rows;
            if (Occupied)
            {
                if (TinC > TPeak) TPeak = TinC;
                DegreeHours += Math.Max(0.0, TinC - TwinConfig.DegreeHourBaseC) * dt;
                OccMin += TwinConfig.StepMinutes;
                ComfortSum += FcNow * TwinConfig.StepMinutes;
                if (allInBand) ComfortMin += TwinConfig.StepMinutes;
                UtilSum += 100.0 * laSum / TwinConfig.Rows * TwinConfig.StepMinutes;
            }
            double d = ThetaDeg - prevTheta;
            Travel += Math.Abs(d);
            if (Math.Abs(d) > 0.5)
            {
                int dir = d > 0 ? 1 : -1;
                if (lastDir != 0 && dir != lastDir) Reversals++;
                lastDir = dir;
            }
            prevTheta = ThetaDeg;

            // 6. physics integration (explicit Euler, after decisions)
```

```
            double teq = ToutC + w.DTGainC * SunFactor * TauBlind;
            TinC += (teq - TinC) * (dt / TwinConfig.ThermalTauH);
            double aqEq = w.AqiOut + (Occupied ? TwinConfig.AqiOccupancyGain : 0.0);
            Aqi += (aqEq - Aqi) * (dt / TwinConfig.AirTauH);

            // 7. advance clock
            MinuteOfDay += TwinConfig.StepMinutes;
            if (MinuteOfDay >= TwinConfig.MinutesPerDay)
            {
                MinuteOfDay -= TwinConfig.MinutesPerDay;
                lastOccupiedMinute -= TwinConfig.MinutesPerDay;
            }
        }

        double LightingLevel(int row, double h)
        {
            switch (Scenario)
            {
                case 0: return 1.0;
                case 1: return (h >= 9.0 && h < 18.0) ? 1.0 : 0.0;
                case 2: return PirActive ? 1.0 : 0.0;
                case 3: return TwinConfig.FixtureDimW / TwinConfig.FixtureFullW;
                case 4: return (row == 0 && h >= 9.0 && h < 13.0) ? 0.0 : 1.0;
                case 5:
                case 6:
                    if (!PirActive) return 0.0;
                    double lpct = Math.Min(100.0, EdayLux[row] / TwinConfig.LdrFullScaleLux * 100.0);
                    double y = lightFis.Evaluate(new[] { lpct, h, TinC }, LightActs);
                    return double.IsNaN(y) ? 0.0 : y / 100.0;
                default: return 1.0;
            }
        }

        // ================ results ================
        public DayResult Snapshot()
        {
            DayResult r = new DayResult();
            r.Scenario = Scenario; r.Weather = Weather;
            r.LightingKWh = LightingWh / 1000.0;
            r.CoolingKWh = CoolingWh / 1000.0;
            r.TotalKWh = r.LightingKWh + r.CoolingKWh;
            r.OccupiedMinutes = OccMin; r.ComfortMinutes = ComfortMin; r.HeldSteps = Held; r.Reversals = Reversals;
            r.ComfortBandPct = OccMin > 0 ? 100.0 * ComfortMin / OccMin : 0.0;
            r.FcAvg = OccMin > 0 ? ComfortSum / OccMin : 0.0;
            r.UAvgPct = OccMin > 0 ? UtilSum / OccMin : 0.0;
            r.ReducedArtificialPct = 100.0 - r.UAvgPct;          // IoT-J 2026 Eq. (12)
            r.TPeakC = TPeak; r.DegreeHours28 = DegreeHours; r.TravelDeg = Travel;
            return r;
        }

        /// <summary>Deterministic 24-h run (00:00 -> 24:00) from a fresh state.</summary>
        public static DayResult RunDay(int scenario, Weather weather, bool paperSlots = false)
        {
            TwinSimulator s = new TwinSimulator(scenario, weather, paperSlots);
            for (int i = 0; i < TwinConfig.MinutesPerDay / TwinConfig.StepMinutes; i++) s.Step();
            return s.Snapshot();
        }

        public static DayResult[,] RunAll()
        {
            int nw = TwinConfig.Profiles.Length;
            DayResult[,] res = new DayResult[nw, 7];
            for (int w = 0; w < nw; w++)
                for (int s = 0; s < 7; s++)
                    res[w, s] = RunDay(s, (Weather)w);
            return res;
        }

        public static string FormatTable(DayResult[,] res)
        {
            System.Text.StringBuilder sb = new System.Text.StringBuilder();
            System.Globalization.CultureInfo ci = System.Globalization.CultureInfo.InvariantCulture;
            sb.AppendLine("weather,scenario,lighting_kWh,cooling_kWh,total_kWh,save_vs_S0_pct,save_vs_S2_pct,"
                        + "total_save_vs_S2_pct,comfort_band_pct,Fc_avg,U_avg_pct,reduced_artificial_pct,"
                        + "Tpeak_C,degree_hours_28,reversals,travel_deg,held_steps");
            for (int w = 0; w < res.GetLength(0); w++)
            {
                double e0 = res[w, 0].LightingKWh, e2 = res[w, 2].LightingKWh, t2 = res[w, 2].TotalKWh;
                for (int s = 0; s < res.GetLength(1); s++)
                {
                    DayResult r = res[w, s];
                    sb.AppendLine(string.Format(ci, "{0},S{1},{2:F4},{3:F4},{4:F4},{5:F2},{6:F2},{7:F2},{8:F1},{9:F3},{10:F2},{
»» 11:F2},{12:F2},{13:F2},{14},{15:F1},{16}",
                        TwinConfig.Profiles[w].Name, s, r.LightingKWh, r.CoolingKWh, r.TotalKWh,
                        100.0 * (1.0 - r.LightingKWh / e0), 100.0 * (1.0 - r.LightingKWh / e2), 100.0 * (1.0 - r.TotalKWh /
»» t2),
                        r.ComfortBandPct, r.FcAvg, r.UAvgPct, r.ReducedArtificialPct, r.TPeakC, r.DegreeHours28, r.Reversals,
»» r.TravelDeg, r.HeldSteps));
                }
            }
            return sb.ToString();
        }
    }
}
```

Assets/Scripts/Core/TwinSelfTest.cs

```
// ClassroomTwin - self-test. Expected values were generated by the independent Python reference
// model (ref/model.py) used to write the specification. Run from Unity menu: Digital Twin > Run Self-Test.
// Every line must print PASS. Tolerance: 1e-6 absolute (1e-4 for 24-h energy sums).
using System;
using System.Collections.Generic;

namespace ClassroomTwin.Core
{
    public static class TwinSelfTest
    {
        public static int Failures;

        static void Check(List<string> log, string name, double got, double expected)
        {
            double tol = name.StartsWith("Day_") ? 1e-4 : 1e-6;
            bool ok = (double.IsNaN(expected) && double.IsNaN(got)) || Math.Abs(got - expected) <= tol;
            if (!ok) Failures++;
            log.Add((ok ? "PASS " : "FAIL ") + name + "  got=" + got.ToString("R") + "  expected=" + expected.ToString("R"));
        }

        public static List<string> Run()
        {
            Failures = 0;
            List<string> log = new List<string>();
            MamdaniFIS lightPaper = TwinControllers.BuildLightFis(true);
            MamdaniFIS lightClass = TwinControllers.BuildLightFis(false);
            MamdaniFIS curtain = TwinControllers.BuildCurtainFis(false, true);
            DayResult[,] days = TwinSimulator.RunAll();
            Check(log, "MF_Light_Low_0", TwinControllers.LightInput().Sets[0].Mu(0), 1.0);
            Check(log, "MF_Light_Low_25", TwinControllers.LightInput().Sets[0].Mu(25), 1.0);
            Check(log, "MF_Light_Low_35", TwinControllers.LightInput().Sets[0].Mu(35), 0.5);
            Check(log, "MF_Light_Low_45", TwinControllers.LightInput().Sets[0].Mu(45), 0.0);
            Check(log, "MF_Light_Medium_45", TwinControllers.LightInput().Sets[1].Mu(45), 0.5);
            Check(log, "MF_Light_Medium_55", TwinControllers.LightInput().Sets[1].Mu(55), 1.0);
            Check(log, "MF_Light_High_70", TwinControllers.LightInput().Sets[2].Mu(70), 0.5);
            Check(log, "MF_Light_High_100", TwinControllers.LightInput().Sets[2].Mu(100), 1.0);
            Check(log, "Tau_0", TwinSimulator.Tau(0), 1.0);
            Check(log, "Tau_25", TwinSimulator.Tau(25), 0.5734196920554815);
            Check(log, "Tau_45", TwinSimulator.Tau(45), 0.28626409273982867);
            Check(log, "Tau_75", TwinSimulator.Tau(75), 0.050000000000000044);
            Check(log, "ToutNormal_3", TwinSimulator.Tout(3, TwinConfig.Profile(Weather.Normal)), 20.0);
            Check(log, "ToutNormal_9", TwinSimulator.Tout(9, TwinConfig.Profile(Weather.Normal)), 24.0);
            Check(log, "ToutNormal_15", TwinSimulator.Tout(15, TwinConfig.Profile(Weather.Normal)), 28.0);
            Check(log, "Sun_6", TwinSimulator.Sun(6), 0.0);
            Check(log, "Sun_9", TwinSimulator.Sun(9), 0.7071067811865475);
            Check(log, "Sun_12", TwinSimulator.Sun(12), 1.0);
            Check(log, "LightFisPaper_4_19_30", lightPaper.Evaluate(new double[] { 4, 19, 30 }, null), 27.727272727272798);
            Check(log, "LightFisPaper_9_11_18", lightPaper.Evaluate(new double[] { 9, 11, 18 }, null), 59.99999999999986);
            Check(log, "LightFisPaper_20_12_22", lightPaper.Evaluate(new double[] { 20, 12, 22 }, null), 50.0);
            Check(log, "LightFisPaper_30_19_34", lightPaper.Evaluate(new double[] { 30, 19, 34 }, null), 27.727272727272798);
            Check(log, "LightFisPaper_45_4_23", lightPaper.Evaluate(new double[] { 45, 4, 23 }, null), 19.999999999999996);
            Check(log, "LightFisPaper_65_10_25", lightPaper.Evaluate(new double[] { 65, 10, 25 }, null), 53.56643356643342);
            Check(log, "LightFisPaper_87_14_30", lightPaper.Evaluate(new double[] { 87, 14, 30 }, null), 20.000000000000004);
            Check(log, "LightFisClass_10_12_26", lightClass.Evaluate(new double[] { 10, 12, 26 }, null), 49.99999999999996);
            Check(log, "LightFisClass_40_11_27", lightClass.Evaluate(new double[] { 40, 11, 27 }, null), 64.99999999999999);
            Check(log, "LightFisClass_90_12_29", lightClass.Evaluate(new double[] { 90, 12, 29 }, null), 20.000000000000007);
            Check(log, "LightFisClass_100_12_42", lightClass.Evaluate(new double[] { 100, 12, 42 }, null), 20.000000000000004);
            Check(log, "CurtainFis_100_35_105_12_0", curtain.Evaluate(new double[] { 100, 35, 105, 12, 0 }, null), 72.
»» 89285714285715);
            Check(log, "CurtainFis_100_25_50_11_0", curtain.Evaluate(new double[] { 100, 25, 50, 11, 0 }, null), 0.0);
            Check(log, "CurtainFis_50_25_50_16_0", curtain.Evaluate(new double[] { 50, 25, 50, 16, 0 }, null), 45.
»» 00000000000001);
            Check(log, "CurtainFis_70_24_300_10_45", curtain.Evaluate(new double[] { 70, 24, 300, 10, 45 }, null), 44.
»» 99999999999999);
            Check(log, "CurtainFis_10_25_300_12_0", curtain.Evaluate(new double[] { 10, 25, 300, 12, 0 }, null), double.NaN);
            Check(log, "CurtainFis_70_23_60_9_75", curtain.Evaluate(new double[] { 70, 23, 60, 9, 75 }, null), 25.
»» 833333333333332);
            Check(log, "Day_Normal_S0_LightingKWh", days[0, 0].LightingKWh, 18.72);
            Check(log, "Day_Normal_S0_CoolingKWh", days[0, 0].CoolingKWh, 9.612414889571395);
            Check(log, "Day_Normal_S0_TPeakC", days[0, 0].TPeakC, 32.05693030435296);
            Check(log, "Day_Normal_S1_LightingKWh", days[0, 1].LightingKWh, 7.02);
            Check(log, "Day_Normal_S1_CoolingKWh", days[0, 1].CoolingKWh, 9.612414889571395);
            Check(log, "Day_Normal_S1_TPeakC", days[0, 1].TPeakC, 32.05693030435296);
            Check(log, "Day_Normal_S2_LightingKWh", days[0, 2].LightingKWh, 6.5);
            Check(log, "Day_Normal_S2_CoolingKWh", days[0, 2].CoolingKWh, 9.612414889571395);
            Check(log, "Day_Normal_S2_TPeakC", days[0, 2].TPeakC, 32.05693030435296);
            Check(log, "Day_Normal_S3_LightingKWh", days[0, 3].LightingKWh, 9.791999999999984);
            Check(log, "Day_Normal_S3_CoolingKWh", days[0, 3].CoolingKWh, 8.62041488957139);
            Check(log, "Day_Normal_S3_TPeakC", days[0, 3].TPeakC, 32.05693030435296);
            Check(log, "Day_Normal_S4_LightingKWh", days[0, 4].LightingKWh, 17.67999999999998);
            Check(log, "Day_Normal_S4_CoolingKWh", days[0, 4].CoolingKWh, 9.265748222904735);
            Check(log, "Day_Normal_S4_TPeakC", days[0, 4].TPeakC, 32.05693030435296);
            Check(log, "Day_Normal_S5_LightingKWh", days[0, 5].LightingKWh, 2.7748727057675673);
            Check(log, "Day_Normal_S5_CoolingKWh", days[0, 5].CoolingKWh, 8.425263448508176);
            Check(log, "Day_Normal_S5_TPeakC", days[0, 5].TPeakC, 32.05693030435296);
            Check(log, "Day_Normal_S6_LightingKWh", days[0, 6].LightingKWh, 3.068612726602627);
            Check(log, "Day_Normal_S6_CoolingKWh", days[0, 6].CoolingKWh, 5.954068474424943);
            Check(log, "Day_Normal_S6_TPeakC", days[0, 6].TPeakC, 29.504692418913137);
            Check(log, "Day_Heatwave_S0_LightingKWh", days[1, 0].LightingKWh, 18.72);
            Check(log, "Day_Heatwave_S0_CoolingKWh", days[1, 0].CoolingKWh, 12.437070473160666);
            Check(log, "Day_Heatwave_S0_TPeakC", days[1, 0].TPeakC, 42.05693030435295);
            Check(log, "Day_Heatwave_S1_LightingKWh", days[1, 1].LightingKWh, 7.02);
            Check(log, "Day_Heatwave_S1_CoolingKWh", days[1, 1].CoolingKWh, 12.437070473160666);
            Check(log, "Day_Heatwave_S1_TPeakC", days[1, 1].TPeakC, 42.05693030435295);
            Check(log, "Day_Heatwave_S2_LightingKWh", days[1, 2].LightingKWh, 6.5);
            Check(log, "Day_Heatwave_S2_CoolingKWh", days[1, 2].CoolingKWh, 12.437070473160666);
            Check(log, "Day_Heatwave_S2_TPeakC", days[1, 2].TPeakC, 42.05693030435295);
```

```
            Check(log, "Day_Heatwave_S3_LightingKWh", days[1, 3].LightingKWh, 9.791999999999984);
            Check(log, "Day_Heatwave_S3_CoolingKWh", days[1, 3].CoolingKWh, 11.445070473160651);
            Check(log, "Day_Heatwave_S3_TPeakC", days[1, 3].TPeakC, 42.05693030435295);
            Check(log, "Day_Heatwave_S4_LightingKWh", days[1, 4].LightingKWh, 17.67999999999998);
            Check(log, "Day_Heatwave_S4_CoolingKWh", days[1, 4].CoolingKWh, 12.090403806493999);
            Check(log, "Day_Heatwave_S4_TPeakC", days[1, 4].TPeakC, 42.05693030435295);
            Check(log, "Day_Heatwave_S5_LightingKWh", days[1, 5].LightingKWh, 2.6979715012322028);
            Check(log, "Day_Heatwave_S5_CoolingKWh", days[1, 5].CoolingKWh, 11.224285297252342);
            Check(log, "Day_Heatwave_S5_TPeakC", days[1, 5].TPeakC, 42.05693030435295);
            Check(log, "Day_Heatwave_S6_LightingKWh", days[1, 6].LightingKWh, 3.165514689822658);
            Check(log, "Day_Heatwave_S6_CoolingKWh", days[1, 6].CoolingKWh, 2.532458185617115);
            Check(log, "Day_Heatwave_S6_TPeakC", days[1, 6].TPeakC, 38.43179482017445);
            Check(log, "Day_Pollution_S0_LightingKWh", days[2, 0].LightingKWh, 18.72);
            Check(log, "Day_Pollution_S0_CoolingKWh", days[2, 0].CoolingKWh, 9.612414889571395);
            Check(log, "Day_Pollution_S0_TPeakC", days[2, 0].TPeakC, 32.05693030435296);
            Check(log, "Day_Pollution_S1_LightingKWh", days[2, 1].LightingKWh, 7.02);
            Check(log, "Day_Pollution_S1_CoolingKWh", days[2, 1].CoolingKWh, 9.612414889571395);
            Check(log, "Day_Pollution_S1_TPeakC", days[2, 1].TPeakC, 32.05693030435296);
            Check(log, "Day_Pollution_S2_LightingKWh", days[2, 2].LightingKWh, 6.5);
            Check(log, "Day_Pollution_S2_CoolingKWh", days[2, 2].CoolingKWh, 9.612414889571395);
            Check(log, "Day_Pollution_S2_TPeakC", days[2, 2].TPeakC, 32.05693030435296);
            Check(log, "Day_Pollution_S3_LightingKWh", days[2, 3].LightingKWh, 9.791999999999984);
            Check(log, "Day_Pollution_S3_CoolingKWh", days[2, 3].CoolingKWh, 8.62041488957139);
            Check(log, "Day_Pollution_S3_TPeakC", days[2, 3].TPeakC, 32.05693030435296);
            Check(log, "Day_Pollution_S4_LightingKWh", days[2, 4].LightingKWh, 17.67999999999998);
            Check(log, "Day_Pollution_S4_CoolingKWh", days[2, 4].CoolingKWh, 9.265748222904735);
            Check(log, "Day_Pollution_S4_TPeakC", days[2, 4].TPeakC, 32.05693030435296);
            Check(log, "Day_Pollution_S5_LightingKWh", days[2, 5].LightingKWh, 2.7748727057675673);
            Check(log, "Day_Pollution_S5_CoolingKWh", days[2, 5].CoolingKWh, 8.425263448508176);
            Check(log, "Day_Pollution_S5_TPeakC", days[2, 5].TPeakC, 32.05693030435296);
            Check(log, "Day_Pollution_S6_LightingKWh", days[2, 6].LightingKWh, 3.1623117468577484);
            Check(log, "Day_Pollution_S6_CoolingKWh", days[2, 6].CoolingKWh, 3.0873282948606846);
            Check(log, "Day_Pollution_S6_TPeakC", days[2, 6].TPeakC, 28.68623248583292);
            log.Add(Failures == 0 ? "SELF-TEST PASSED (" + log.Count + " checks)" : "SELF-TEST FAILED: " + Failures + " of " +
»» log.Count + " checks");
            return log;
        }
    }
}
```

Assets/Scripts/SimulationEngine.cs

```
// ClassroomTwin - runtime bridge between the deterministic TwinSimulator and the Unity scene.
// The simulator owns ALL physics and control decisions. This MonoBehaviour only:
//   (1) advances the simulator in fixed 1-minute steps,  (2) maps its state to slats/lights/sun,
//   (3) handles keys,  (4) draws the HUD and the 24-h batch table.
using System.Collections.Generic;
using System.IO;
using UnityEngine;
using ClassroomTwin.Core;

public class SimulationEngine : MonoBehaviour
{
    [Header("Simulation")]
    [Range(0, 6)] public int scenario = 6;
    public Weather weather = Weather.Normal;
    [Tooltip("Simulated minutes per real second (15 => one day in 96 s)")]
    public float minutesPerRealSecond = 15f;
    [Tooltip("Start-of-run clock, minutes after midnight (510 = 08:30)")]
    public int startMinute = 510;

    [Header("Visual mapping (cosmetic only, never affects results)")]
    public float servoVisualSpeedDegPerSec = 40f;
    public float spotIntensityAtFull = 1.0f;
    public float sunIntensityAtNoon = 1.2f;
    public bool showHud = true;

    TwinSimulator sim;
    float minuteAccumulator;
    bool paused;
    float visualAngle;
    bool showBatch;
    string batchText = "";

    readonly List<Transform> slats = new List<Transform>();
    readonly List<Light>[] rowLights = { new List<Light>(), new List<Light>(), new List<Light>() };
    readonly List<Renderer>[] rowPanels = { new List<Renderer>(), new List<Renderer>(), new List<Renderer>() };
    Light sun;
    MaterialPropertyBlock mpb;
    GUIStyle boxStyle, labelStyle, headStyle, goodStyle, hintStyle, monoStyle;

    const int MaxStepsPerFrame = 240;
    static readonly string[] RowObjectNames = { "Row1_Window", "Row2_Center", "Row3_Corridor" };
    static readonly int EmissionId = Shader.PropertyToID("_EmissionColor");

    void Start()
    {
        QualitySettings.pixelLightCount = Mathf.Max(QualitySettings.pixelLightCount, 16);
        mpb = new MaterialPropertyBlock();
        FindHardware();
        sim = new TwinSimulator(scenario, weather);
        sim.ResetToMinute(startMinute);
        visualAngle = (float)sim.ThetaDeg;
        ApplyVisuals(0f);
    }

    void FindHardware()
```

```
    {
        GameObject lighting = GameObject.Find("Lighting_System");
        if (lighting == null) Debug.LogError("SimulationEngine: 'Lighting_System' not found - run Digital Twin > Generate
»» Classroom first.");
        else
        {
            for (int r = 0; r < 3; r++)
            {
                Transform row = lighting.transform.Find(RowObjectNames[r]);
                if (row == null) { Debug.LogError("SimulationEngine: missing " + RowObjectNames[r]); continue; }
                foreach (Transform fixture in row)
                {
                    Light l = fixture.GetComponentInChildren<Light>();
                    Renderer rend = fixture.GetComponent<Renderer>();
                    if (l != null) rowLights[r].Add(l);
                    if (rend != null) rowPanels[r].Add(rend);
                }
            }
        }

        GameObject blinds = GameObject.Find("Smart_Blinds");
        if (blinds == null) Debug.LogError("SimulationEngine: 'Smart_Blinds' not found - regenerate the classroom.");
        else
            foreach (Transform bay in blinds.transform)
                foreach (Transform child in bay)
                    if (child.name.StartsWith("Slat_")) slats.Add(child);

        GameObject sunGo = GameObject.Find("Sunlight");
        if (sunGo != null) sun = sunGo.GetComponent<Light>();

        if (slats.Count != 30) Debug.LogWarning("SimulationEngine: expected 30 slats, found " + slats.Count);
        int lights = rowLights[0].Count + rowLights[1].Count + rowLights[2].Count;
        if (lights != 12) Debug.LogWarning("SimulationEngine: expected 12 fixtures, found " + lights);
    }

    void Update()
    {
        HandleKeys();
        if (!paused)
        {
            minuteAccumulator += Time.deltaTime * minutesPerRealSecond;
            int steps = (int)minuteAccumulator;
            if (steps > MaxStepsPerFrame) { steps = MaxStepsPerFrame; minuteAccumulator = 0f; }
            else minuteAccumulator -= steps;
            for (int i = 0; i < steps; i++) sim.Step();
        }
        ApplyVisuals(Time.deltaTime);
    }

    void HandleKeys()
    {
        for (int s = 0; s <= 6; s++)
            if (Input.GetKeyDown(KeyCode.Alpha0 + s) || Input.GetKeyDown(KeyCode.Keypad0 + s)) { scenario = s; sim.SetScenario(
»» s); }

        if (Input.GetKeyDown(KeyCode.T)) { weather = (weather == Weather.Heatwave) ? Weather.Normal : Weather.Heatwave; sim.
»» Weather = weather; }
        if (Input.GetKeyDown(KeyCode.Q)) { weather = (weather == Weather.Pollution) ? Weather.Normal : Weather.Pollution; sim.
»» Weather = weather; }
        if (Input.GetKeyDown(KeyCode.Space)) paused = !paused;
        if (Input.GetKeyDown(KeyCode.R)) { sim.ResetToMinute(startMinute); minuteAccumulator = 0f; }
        if (Input.GetKeyDown(KeyCode.RightBracket)) minutesPerRealSecond = Mathf.Min(120f, minutesPerRealSecond * 2f);
        if (Input.GetKeyDown(KeyCode.LeftBracket)) minutesPerRealSecond = Mathf.Max(1f, minutesPerRealSecond * 0.5f);
        if (Input.GetKeyDown(KeyCode.H)) showHud = !showHud;
        if (Input.GetKeyDown(KeyCode.B))
        {
            showBatch = !showBatch;
            if (showBatch) RunBatch();
        }
    }

    void RunBatch()
    {
        DayResult[,] res = TwinSimulator.RunAll();
        string csv = TwinSimulator.FormatTable(res);
        string path = Path.Combine(Application.persistentDataPath, "TwinResults.csv");
        File.WriteAllText(path, csv);
        Debug.Log("ClassroomTwin batch written to " + path + "\n" + csv);

        System.Text.StringBuilder sb = new System.Text.StringBuilder();
        sb.AppendLine("24-h deterministic batch (lighting kWh | cooling kWh | total kWh | light save vs S0 | Tpeak C)");
        for (int w = 0; w < res.GetLength(0); w++)
        {
            sb.AppendLine("--- " + TwinConfig.Profiles[w].Name + " ---");
            double e0 = res[w, 0].LightingKWh;
            for (int s = 0; s < 7; s++)
            {
                DayResult r = res[w, s];
                sb.AppendLine(string.Format(System.Globalization.CultureInfo.InvariantCulture,
                    "S{0}  {1,7:F3}  {2,7:F3}  {3,7:F3}  {4,6:F1}%  {5,5:F1}", s, r.LightingKWh, r.CoolingKWh, r.TotalKWh, 100.
»» 0 * (1.0 - r.LightingKWh / e0), r.TPeakC));
            }
        }
        batchText = sb.ToString();
    }

    void ApplyVisuals(float dt)
```

```
    {
        // 1. slats - cosmetic servo animation towards the modelled angle
        visualAngle = (dt <= 0f) ? (float)sim.ThetaDeg : Mathf.MoveTowards(visualAngle, (float)sim.ThetaDeg,
»» servoVisualSpeedDegPerSec * dt);
        Quaternion rot = Quaternion.Euler(visualAngle, 0f, 0f);
        for (int i = 0; i < slats.Count; i++) slats[i].localRotation = rot;

        // 2. LED rows - PWM level drives light intensity and panel emission
        for (int r = 0; r < 3; r++)
        {
            float la = (float)sim.La[r];
            for (int i = 0; i < rowLights[r].Count; i++)
            {
                rowLights[r][i].enabled = la > 0.001f;
                rowLights[r][i].intensity = spotIntensityAtFull * la;
            }
            for (int i = 0; i < rowPanels[r].Count; i++)
            {
                rowPanels[r][i].GetPropertyBlock(mpb);
                mpb.SetColor(EmissionId, Color.white * (2.0f * la));
                rowPanels[r][i].SetPropertyBlock(mpb);
            }
        }

        // 3. sun - smooth clock; always enters through the window wall (light travels towards +Z)
        if (sun != null)
        {
            float h = (sim.MinuteOfDay + minuteAccumulator) / 60f;
            float sf = (float)TwinSimulator.Sun(h);
            sun.enabled = sf > 0f;
            if (sf > 0f)
            {
                float elevation = 5f + 55f * sf;                // 5 deg at sunrise/sunset, 60 deg at noon
                float azimuth = -60f + 120f * (h - 6f) / 12f;    // -60 deg (06:00) -> +60 deg (18:00)
                sun.transform.rotation = Quaternion.Euler(elevation, azimuth, 0f);
                sun.intensity = sunIntensityAtNoon * sf * (float)(TwinConfig.Profile(weather).EdiffMaxLux / 20000.0);
            }
        }
    }

    // ---------------- HUD ----------------
    void BuildStyles()
    {
        if (boxStyle != null) return;
        boxStyle = new GUIStyle(GUI.skin.box) { fontSize = 18, fontStyle = FontStyle.Bold, alignment = TextAnchor.UpperLeft };
        boxStyle.normal.textColor = Color.white;
        labelStyle = new GUIStyle(GUI.skin.label) { fontSize = 14 };
        labelStyle.normal.textColor = Color.white;
        headStyle = new GUIStyle(labelStyle) { fontStyle = FontStyle.Bold };
        headStyle.normal.textColor = new Color(0.6f, 0.85f, 1f);
        goodStyle = new GUIStyle(labelStyle) { fontSize = 16, fontStyle = FontStyle.Bold };
        goodStyle.normal.textColor = Color.green;
        hintStyle = new GUIStyle(labelStyle) { fontSize = 12 };
        hintStyle.normal.textColor = Color.yellow;
        monoStyle = new GUIStyle(labelStyle) { fontSize = 13 };
        monoStyle.font = Font.CreateDynamicFontFromOSFont("Courier New", 13);
    }

    static string AqiBand(double a)
    {
        if (a <= 50) return "Good";
        if (a <= 100) return "Satisfactory";
        if (a <= 200) return "Moderate";
        if (a <= 300) return "Poor";
        if (a <= 400) return "Very Poor";
        return "Severe";
    }

    static string CurtainLabel(double t) { return t < 15.0 ? "Open" : (t < 60.0 ? "Partial" : "Closed"); }

    void OnGUI()
    {
        if (sim == null || !showHud) return;
        BuildStyles();

        int m = sim.MinuteOfDay;
        string clock = string.Format("{0:00}:{1:00}", m / 60, m % 60);
        float x = 10f, y = 10f, w = 560f, lh = 21f;
        GUI.Box(new Rect(x, y, w, 430f), "Classroom Digital Twin - Fuzzy Co-Regulation", boxStyle);
        y += 34f;

        Line(ref y, x, w, lh, "Time " + clock + "   |   " + minutesPerRealSecond.ToString("0") + " sim-min / s" + (paused ? "
»»  [PAUSED]" : ""), labelStyle);
        Line(ref y, x, w, lh, "Mode [" + sim.Scenario + "] " + TwinConfig.ScenarioNames[sim.Scenario].Substring(3), headStyle);
        Line(ref y, x, w, lh, "Weather: " + TwinConfig.Profile(sim.Weather).Name + "   |   Room: " + (sim.Occupied ?
»» "Occupied" : (sim.PirActive ? "Empty (PIR hold)" : "Empty")), labelStyle);
        y += 4f;
        Line(ref y, x, w, lh, "SENSORS", headStyle);
        Line(ref y, x, w, lh, string.Format("Ext. diffuse {0:F0} lux | LDR(window) {1:F0} %", sim.EdiffLux, sim.LdrCurtainPct),
»»  labelStyle);
        Line(ref y, x, w, lh, string.Format("T_out {0:F1} C | T_in {1:F1} C | AQI {2:F0} ({3})", sim.ToutC, sim.TinC, sim.Aqi,
»» AqiBand(sim.Aqi)), labelStyle);
        y += 4f;
        Line(ref y, x, w, lh, "ACTUATORS", headStyle);
        Line(ref y, x, w, lh, string.Format("Blinds {0:F1} deg [{1}]  (servo view {2:F1} deg) | transmission {3:F2}{4}",
            sim.ThetaDeg, CurtainLabel(sim.ThetaDeg), visualAngle, sim.TauBlind, sim.CurtainHeld ? " | HOLD" : ""),
»» labelStyle);
```

```
        for (int r = 0; r < 3; r++)
            Line(ref y, x, w, lh, string.Format("{0,-8} LED {1,3:F0} %  daylight {2,5:F0} lux  task {3,5:F0} lux",
                TwinConfig.RowNames[r], sim.La[r] * 100.0, sim.EdayLux[r], sim.EtaskLux[r]), labelStyle);
        y += 4f;
        Line(ref y, x, w, lh, "ENERGY", headStyle);
        Line(ref y, x, w, lh, string.Format("Lighting power {0:F0} W  (baseline {1:F0} W) | visual comfort Fc {2:F2}", sim.
»» PowerW, TwinConfig.BaselinePowerW, sim.FcNow), labelStyle);
        double save = sim.BaselineWh > 0 ? 100.0 * (1.0 - sim.LightingWh / sim.BaselineWh) : 0.0;
        Line(ref y, x, w, lh, string.Format("Since last switch: {0:F2} kWh lighting vs {1:F2} kWh baseline | cooling est. {2:
»» F2} kWh",
            sim.LightingWh / 1000.0, sim.BaselineWh / 1000.0, sim.CoolingWh / 1000.0), labelStyle);
        Line(ref y, x, w, lh + 3f, string.Format("Lighting energy saved: {0:F1} %", save), goodStyle);
        Line(ref y, x, w, lh, "0-6 scenario | T heatwave | Q pollution | B 24-h batch | [ ] speed | Space pause | R reset | H
»» hide", hintStyle);

        if (showBatch && batchText.Length > 0)
        {
            GUI.Box(new Rect(Screen.width - 600f, 10f, 590f, 560f), "", boxStyle);
            GUI.Label(new Rect(Screen.width - 590f, 18f, 575f, 545f), batchText, monoStyle);
        }
    }

    static void Line(ref float y, float x, float w, float h, string text, GUIStyle style)
    {
        GUI.Label(new Rect(x + 10f, y, w - 20f, h), text, style);
        y += h;
    }
}
```

Assets/Editor/ClassroomBuilder.cs

```
// ClassroomTwin - procedural scene builder (Editor only).
// Menu: Digital Twin > Generate Classroom (Paper Figure 5 Type B)
// Changes vs mid-sem: ceiling added, desks seated on the floor, window bay centres fixed to +/-3.80 m,
// glass panes, 30 interior venetian slats, 3 blind motor units, materials saved as assets,
// LED spot shadows off, sun aimed THROUGH the window wall, Undo + scene-dirty support.
using UnityEngine;
using UnityEngine.Rendering;
using UnityEngine.SceneManagement;
using UnityEditor;
using UnityEditor.SceneManagement;

public static class ClassroomBuilder
{
    const string MatFolder = "Assets/Materials/Twin";

    // ---------- geometry constants (metres) ----------
    public const float WindowWallZ = -4.4f;          // centre plane of window wall (thickness 0.2 => inner face -4.3)
    public const float SillTopY = 0.8f;
    public const float HeaderBottomY = 2.4f;
    public static readonly float[] BayCentersX = { -3.80f, 0.0f, 3.80f };
    public const float BayWidth = 3.50f;
    public const float SlatZ = -4.20f;               // interior side, 0.10 m in front of inner wall face
    public const float SlatLength = 3.46f;           // 0.02 m clearance each side
    public const float SlatThickness = 0.012f;
    public const float SlatDepth = 0.17f;            // must equal TwinConfig.SlatDepthM
    public const float SlatPitch = 0.16f;            // must equal TwinConfig.SlatPitchM
    public const int SlatsPerBay = 10;
    public const float FirstSlatY = 0.88f;           // 0.8 + pitch/2  -> slats at 0.88 ... 2.32

    [MenuItem("Digital Twin/Generate Classroom (Paper Figure 5 Type B)")]
    public static void GenerateClassroom()
    {
        bool urp = GraphicsSettings.currentRenderPipeline != null;
        Shader lit = Shader.Find(urp ? "Universal Render Pipeline/Lit" : "Standard");
        if (lit == null) { Debug.LogError("ClassroomBuilder: lit shader not found. Use the Built-in Render Pipeline (Standard
»» shader)."); return; }
        if (!AssetDatabase.IsValidFolder("Assets/Materials")) AssetDatabase.CreateFolder("Assets", "Materials");
        if (!AssetDatabase.IsValidFolder(MatFolder)) AssetDatabase.CreateFolder("Assets/Materials", "Twin");

        // 1. Materials (saved as assets so they survive scene save/reload)
        Material matFloor = Mat("Twin_Floor", lit, new Color(0.50f, 0.50f, 0.50f));
        Material matWall = Mat("Twin_Wall", lit, new Color(0.90f, 0.90f, 0.90f));
        Material matCeiling = Mat("Twin_Ceiling", lit, new Color(0.95f, 0.95f, 0.95f));
        Material matBoard = Mat("Twin_Blackboard", lit, new Color(0.10f, 0.30f, 0.10f));
        Material matDesk = Mat("Twin_Desk", lit, new Color(0.60f, 0.40f, 0.20f));
        Material matSlat = Mat("Twin_Slat", lit, new Color(0.90f, 0.92f, 0.95f));
        Material matMotor = Mat("Twin_Motor", lit, new Color(0.15f, 0.15f, 0.18f));
        Material matGlass = Mat("Twin_Glass", lit, new Color(0.70f, 0.90f, 1.00f, 0.25f));
        MakeTransparent(matGlass, urp);
        Material matLED = Mat("Twin_LED", lit, Color.white);
        matLED.EnableKeyword("_EMISSION");
        matLED.globalIlluminationFlags = MaterialGlobalIlluminationFlags.RealtimeEmissive;
        matLED.SetColor("_EmissionColor", Color.white * 2.0f);
        AssetDatabase.SaveAssets();

        // 2. Root (replace previous build)
        GameObject old = GameObject.Find("Classroom_Room302");
        if (old != null) Undo.DestroyObjectImmediate(old);
        GameObject room = new GameObject("Classroom_Room302");
        Undo.RegisterCreatedObjectUndo(room, "Generate Classroom");

        // 3. Floor (top surface at y = 0.10) and ceiling (bottom surface at y = 3.00)
        Box("Floor", room.transform, new Vector3(0f, 0f, 0f), new Vector3(11.5f, 0.2f, 9.0f), matFloor);
        Box("Ceiling", room.transform, new Vector3(0f, 3.05f, 0f), new Vector3(11.5f, 0.1f, 9.0f), matCeiling);

        // 4. Walls (height 3.0, y 0..3)
```

```
        Box("Wall_Back", room.transform, new Vector3(5.65f, 1.5f, 0f), new Vector3(0.2f, 3.0f, 9.0f), matWall);
        Box("Wall_Front", room.transform, new Vector3(-5.65f, 1.5f, 0f), new Vector3(0.2f, 3.0f, 9.0f), matWall);
        Box("Wall_Corridor", room.transform, new Vector3(0f, 1.5f, 4.4f), new Vector3(11.5f, 3.0f, 0.2f), matWall);
        Box("Wall_Window_Sill", room.transform, new Vector3(0f, 0.4f, WindowWallZ), new Vector3(11.5f, 0.8f, 0.2f), matWall);
        Box("Wall_Window_Header", room.transform, new Vector3(0f, 2.7f, WindowWallZ), new Vector3(11.5f, 0.6f, 0.2f), matWall);
        for (int i = 0; i < 2; i++)
            Box("Pillar_" + (i + 1), room.transform, new Vector3(i == 0 ? -1.9f : 1.9f, 1.5f, WindowWallZ), new Vector3(0.3f,
»» 3.0f, 0.25f), matWall);

        // 5. Glass panes (do not cast shadows, no colliders)
        GameObject glass = new GameObject("Window_Glass");
        glass.transform.parent = room.transform;
        for (int b = 0; b < 3; b++)
        {
            GameObject g = Box("Glass_Bay_" + (b + 1), glass.transform, new Vector3(BayCentersX[b], 1.6f, WindowWallZ), new
»» Vector3(BayWidth, 1.6f, 0.01f), matGlass);
            Object.DestroyImmediate(g.GetComponent<Collider>());
            MeshRenderer mr = g.GetComponent<MeshRenderer>();
            mr.shadowCastingMode = ShadowCastingMode.Off;
            mr.receiveShadows = false;
        }

        // 6. Podium & blackboard
        Box("Teacher_Podium", room.transform, new Vector3(-4.7f, 0.1f, 0f), new Vector3(1.5f, 0.2f, 6.0f), matDesk);
        Box("Blackboard", room.transform, new Vector3(-5.5f, 1.6f, 0f), new Vector3(0.05f, 1.2f, 4.5f), matBoard);

        // 7. Lighting system - 12 LED panels in 3 rows (names are read by SimulationEngine)
        GameObject lighting = new GameObject("Lighting_System");
        lighting.transform.parent = room.transform;
        float[] xs = { -3.6f, -1.2f, 1.2f, 3.6f };
        float[] zs = { -2.8f, 0.0f, 2.8f };
        string[] rowNames = { "Row1_Window", "Row2_Center", "Row3_Corridor" };
        for (int r = 0; r < 3; r++)
        {
            GameObject row = new GameObject(rowNames[r]);
            row.transform.parent = lighting.transform;
            for (int x = 0; x < 4; x++)
            {
                GameObject panel = Box("LED_Fixture_" + (x + 1), row.transform, new Vector3(xs[x], 2.95f, zs[r]), new Vector3(
»» 1.2f, 0.05f, 0.3f), matLED);
                panel.GetComponent<MeshRenderer>().shadowCastingMode = ShadowCastingMode.Off;
                GameObject ls = new GameObject("Spotlight");
                ls.transform.parent = panel.transform;
                ls.transform.localPosition = new Vector3(0f, -0.6f, 0f);   // = 0.03 m below the panel in world space
                ls.transform.localRotation = Quaternion.Euler(90f, 0f, 0f); // points straight down
                Light l = ls.AddComponent<Light>();
                l.type = LightType.Spot;
                l.spotAngle = 75f;
                l.range = 6.0f;
                l.intensity = 1.0f;
                l.color = new Color(1f, 0.95f, 0.9f);
                l.shadows = LightShadows.None;                              // 12 shadowed spots are too expensive
                l.renderMode = LightRenderMode.ForcePixel;
            }
        }

        // 8. Student desks (seated on floor top y = 0.10 => centre y = 0.475)
        GameObject desks = new GameObject("Student_Desks");
        desks.transform.parent = room.transform;
        for (int x = -2; x <= 4; x += 2)
            for (int z = -2; z <= 2; z += 2)
                Box("Desk_" + x + "_" + z, desks.transform, new Vector3(x, 0.475f, z), new Vector3(0.8f, 0.75f, 1.5f),
»» matDesk);

        // 9. Sun - travels towards +Z so it enters THROUGH the window wall (SimulationEngine animates it)
        GameObject oldSun = GameObject.Find("Sunlight");
        if (oldSun != null) Undo.DestroyObjectImmediate(oldSun);
        GameObject sun = new GameObject("Sunlight");
        Undo.RegisterCreatedObjectUndo(sun, "Generate Classroom");
        Light sl = sun.AddComponent<Light>();
        sl.type = LightType.Directional;
        sl.color = new Color(1f, 0.95f, 0.8f);
        sl.intensity = 1.2f;
        sl.shadows = LightShadows.Soft;
        sun.transform.rotation = Quaternion.Euler(60f, 0f, 0f);

        // 10. Smart motorised venetian blinds: 3 bays x 10 slats, rotation about local X
        GameObject blinds = new GameObject("Smart_Blinds");
        blinds.transform.parent = room.transform;
        for (int b = 0; b < 3; b++)
        {
            GameObject bay = new GameObject("Blinds_Bay_" + (b + 1));
            bay.transform.parent = blinds.transform;
            bay.transform.position = new Vector3(BayCentersX[b], 0f, SlatZ);
            for (int s = 0; s < SlatsPerBay; s++)
            {
                GameObject slat = GameObject.CreatePrimitive(PrimitiveType.Cube);
                slat.name = "Slat_" + b + "_" + s;
                slat.transform.parent = bay.transform;
                slat.transform.localPosition = new Vector3(0f, FirstSlatY + s * SlatPitch, 0f);
                slat.transform.localRotation = Quaternion.identity;
                slat.transform.localScale = new Vector3(SlatLength, SlatThickness, SlatDepth);
                slat.GetComponent<Renderer>().sharedMaterial = matSlat;
                Object.DestroyImmediate(slat.GetComponent<Collider>());
            }
            // MG996R servo + ZigBee end-device housing (visual only)
            GameObject motor = Box("Blind_Motor_Unit_" + (b + 1), bay.transform, Vector3.zero, new Vector3(0.12f, 0.06f, 0.
```

```
»» 08f), matMotor);
            motor.transform.localPosition = new Vector3(-1.65f, 2.44f, 0f);
            Object.DestroyImmediate(motor.GetComponent<Collider>());
        }

        EditorSceneManager.MarkSceneDirty(SceneManager.GetActiveScene());
        Debug.Log("ClassroomTwin: classroom generated - 12 LED fixtures, 30 venetian slats, 3 glass panes, ceiling, sun. Save
»» the scene (Ctrl/Cmd+S).");
    }

    // ---------- helpers ----------
    static GameObject Box(string name, Transform parent, Vector3 worldPos, Vector3 scale, Material m)
    {
        GameObject g = GameObject.CreatePrimitive(PrimitiveType.Cube);
        g.name = name;
        g.transform.parent = parent;
        g.transform.position = worldPos;
        g.transform.localScale = scale;
        g.GetComponent<Renderer>().sharedMaterial = m;
        return g;
    }

    static Material Mat(string name, Shader shader, Color c)
    {
        string path = MatFolder + "/" + name + ".mat";
        Material m = AssetDatabase.LoadAssetAtPath<Material>(path);
        if (m == null) { m = new Material(shader); AssetDatabase.CreateAsset(m, path); }
        m.shader = shader;
        m.color = c;
        if (m.HasProperty("_BaseColor")) m.SetColor("_BaseColor", c);
        EditorUtility.SetDirty(m);
        return m;
    }

    static void MakeTransparent(Material m, bool urp)
    {
        if (urp)
        {
            m.SetFloat("_Surface", 1f);
            m.SetFloat("_Blend", 0f);
            m.EnableKeyword("_SURFACE_TYPE_TRANSPARENT");
        }
        else
        {
            m.SetFloat("_Mode", 3f);
            m.EnableKeyword("_ALPHAPREMULTIPLY_ON");
            m.DisableKeyword("_ALPHATEST_ON");
            m.DisableKeyword("_ALPHABLEND_ON");
        }
        m.SetInt("_SrcBlend", (int)BlendMode.One);
        m.SetInt("_DstBlend", (int)BlendMode.OneMinusSrcAlpha);
        m.SetInt("_ZWrite", 0);
        m.renderQueue = (int)RenderQueue.Transparent;
        EditorUtility.SetDirty(m);
    }
}
```

Assets/Editor/SimulationBuilder.cs

```
// ClassroomTwin - Editor menus: add the engine + camera, run self-test, run 24-h batch.
using System.IO;
using UnityEngine;
using UnityEngine.SceneManagement;
using UnityEditor;
using UnityEditor.SceneManagement;
using ClassroomTwin.Core;

public static class SimulationBuilder
{
    [MenuItem("Digital Twin/Add Simulation Engine")]
    public static void AddSimulationEngine()
    {
        GameObject go = GameObject.Find("Simulation_Engine");
        if (go == null)
        {
            go = new GameObject("Simulation_Engine");
            Undo.RegisterCreatedObjectUndo(go, "Add Simulation Engine");
        }
        if (go.GetComponent<SimulationEngine>() == null) Undo.AddComponent<SimulationEngine>(go);

        // Presentation camera: back-right corner, looking at the window wall and blackboard.
        Camera cam = Camera.main;
        if (cam == null)
        {
            GameObject cg = new GameObject("Main Camera");
            cg.tag = "MainCamera";
            cam = cg.AddComponent<Camera>();
            Undo.RegisterCreatedObjectUndo(cg, "Add Camera");
        }
        Vector3 pos = new Vector3(5.0f, 2.2f, 3.9f);
        Vector3 target = new Vector3(-2.0f, 1.1f, -3.6f);
        cam.transform.position = pos;
        cam.transform.rotation = Quaternion.LookRotation(target - pos, Vector3.up);
        cam.fieldOfView = 70f;
        cam.nearClipPlane = 0.05f;

        Selection.activeGameObject = go;
```

```
        EditorSceneManager.MarkSceneDirty(SceneManager.GetActiveScene());
        Debug.Log("ClassroomTwin: Simulation_Engine ready. Press Play.");
    }

    [MenuItem("Digital Twin/Run Self-Test")]
    public static void RunSelfTest()
    {
        foreach (string line in TwinSelfTest.Run())
        {
            if (line.StartsWith("FAIL") || line.StartsWith("SELF-TEST FAILED")) Debug.LogError(line);
            else if (line.StartsWith("SELF-TEST")) Debug.Log(line);
        }
        if (TwinSelfTest.Failures > 0) Debug.LogError("ClassroomTwin: self-test failed - do not continue until every check
»» passes.");
    }

    [MenuItem("Digital Twin/Run 24h Batch Evaluation")]
    public static void RunBatch()
    {
        string csv = TwinSimulator.FormatTable(TwinSimulator.RunAll());
        string dir = Path.Combine(Directory.GetParent(Application.dataPath).FullName, "TwinResults");
        Directory.CreateDirectory(dir);
        string path = Path.Combine(dir, "TwinResults.csv");
        File.WriteAllText(path, csv);
        Debug.Log("ClassroomTwin 24-h batch results written to " + path + "\n" + csv);
    }
}
```

Appendix B - Python reference model

Independent implementation used to generate every expected value. Run `python3 model.py` to print the 24-h tables. It is not needed by Unity; keep it in the report repository as evidence of verification.

```
"""Reference model for the ClassroomTwin end-sem upgrade.
Every number in the spec PDF is produced by this file. The C# core must match it.
"""
import math

# ---------------- membership functions ----------------
def trap(x, a, b, c, d):
    # shoulders: a==b -> left shoulder (1 for x<=b), c==d -> right shoulder (1 for x>=c)
    if x < a or x > d:
        return 0.0
    if b <= x <= c:
        return 1.0
    if x < b:
        return 1.0 if b == a else (x - a) / (b - a)
    return 1.0 if d == c else (d - x) / (d - c)

def tri(x, a, b, c):
    return trap(x, a, b, b, c)

def mf(x, p):
    return tri(x, *p) if len(p) == 3 else trap(x, *p)

# ---------------- fuzzy variables ----------------
# Light FIS (CCIS 2024) inputs
LIGHT_IN = {"Low": (0, 0, 25, 45), "Medium": (35, 55, 75), "High": (60, 80, 100, 100)}
TEMP_LFIS = {"Cold": (0, 0, 10, 20), "Warm": (15, 22.5, 30), "Hot": (25, 30, 40, 40)}
SLOT_PAPER = {"Sleep": (0, 0, 5, 10), "Work": (8, 14, 20), "Relax": (18, 20, 24, 24)}
SLOT_CLASS = {"Sleep": (0, 0, 6, 8), "Work": (7, 9, 17, 19), "Relax": (17, 19, 24, 24)}
LIGHT_OUT = {"Low": (-10, 20, 50), "Medium": (20, 50, 80), "High": (50, 80, 110)}
LIGHT_OUT_U = (-10.0, 110.0, 0.5)   # min, max, step  -> 241 samples

# Curtain FIS (IoT-J 2026) inputs
TEMP_CFIS = {"Cold": (0, 0, 16, 22), "Warm": (18, 25, 32), "Hot": (28, 34, 40, 40)}
AQI_IN = {"Clean": (0, 0, 100, 200), "Poor": (100, 200, 500, 500)}
ANGLE = {"Open": (-30, 0, 30), "Partial": (15, 45, 75), "Closed": (45, 75, 105)}
ANGLE_U = (-30.0, 105.0, 0.5)       # 271 samples

# CCIS Table I, 27 rules: (light, slot, temp) -> output
LIGHT_RULES = []
_tab = {
    "Low":    {"Work": ("High", "Medium", "Medium"), "Relax": ("Medium", "Medium", "Low"), "Sleep": ("Low", "Low", "Low")},
    "Medium": {"Work": ("High", "High", "Medium"),   "Relax": ("Medium", "Medium", "Medium"), "Sleep": ("Low", "Low", "Low")},
    "High":   {"Work": ("Medium", "Low", "Low"),     "Relax": ("Medium", "Medium", "Low"), "Sleep": ("Medium", "Low", "Low")},
}
for L in ("Low", "Medium", "High"):
    for S in ("Work", "Relax", "Sleep"):
        for i, T in enumerate(("Cold", "Warm", "Hot")):
            LIGHT_RULES.append((L, S, T, _tab[L][S][i]))

# IoT-J Table I (R1-R11) + extensions (R12-R17). None = Any.
# (light, temp, aqi, slot, prevAngle, out)
CURTAIN_RULES = [
    ("Low", None, None, "Sleep", None, "Closed"),        # R1
    ("Medium", "Cold", "Clean", "Work", "Open", "Open"),  # R2
    ("Medium", "Warm", "Clean", "Work", None, "Partial"), # R3
    ("Medium", None, None, "Relax", None, "Partial"),     # R4
    ("High", "Cold", "Clean", "Work", "Open", "Open"),    # R5
    ("High", "Warm", "Clean", "Work", "Partial", "Open"), # R6
    ("High", "Hot", "Clean", "Relax", None, "Partial"),   # R7
    ("High", None, "Poor", None, None, "Partial"),        # R8
    ("Medium", None, "Poor", None, None, "Partial"),      # R9
    (None, None, None, "Sleep", None, "Closed"),          # R10
    (None, None, None, None, "Closed", "Partial"),        # R11
    # ---- extensions (fill coverage gaps of the paper's table) ----
    ("High", "Hot", None, "Work", None, "Closed"),        # R12 thermal/glare protection
    ("Low", None, "Clean", "Work", None, "Open"),         # R13 harvest scarce daylight
    ("Medium", "Hot", "Clean", "Work", None, "Partial"),  # R14
    ("High", "Cold", "Clean", "Work", None, "Open"),      # R15 (R5 without prev condition)
    ("Medium", "Cold", "Clean", "Work", None, "Open"),    # R16 (R2 without prev condition)
    ("High", "Warm", "Clean", "Work", None, "Open"),      # R17 (R6 without prev condition)
    ("Low", None, None, "Relax", None, "Closed"),         # R18 evening privacy
]

def samples(u):
    lo, hi, st = u
    n = int(round((hi - lo) / st)) + 1
    return [lo + i * st for i in range(n)]

def defuzz(acts, outsets, u):
    # acts: dict out_label -> activation (max over rules)
    num = den = 0.0
    for y in samples(u):
        m = 0.0
        for lab, a in acts.items():
            if a > 0:
                v = min(a, mf(y, outsets[lab]))
                if v > m:
                    m = v
        num += y * m
        den += m
```

```
    if den <= 1e-12:
        return None
    return num / den

def clamp(x, lo, hi):
    return lo if x < lo else hi if x > hi else x

def light_fis(light_pct, hour, temp, slots, rules=None):
    rules = LIGHT_RULES if rules is None else rules
    light_pct = clamp(light_pct, 0.0, 100.0); hour = clamp(hour, 0.0, 24.0); temp = clamp(temp, 0.0, 40.0)
    mL = {k: mf(light_pct, p) for k, p in LIGHT_IN.items()}
    mS = {k: mf(hour, p) for k, p in slots.items()}
    mT = {k: mf(temp, p) for k, p in TEMP_LFIS.items()}
    acts = {"Low": 0.0, "Medium": 0.0, "High": 0.0}
    for L, S, T, o in rules:
        a = min(mL[L], mS[S], mT[T])
        if a > acts[o]:
            acts[o] = a
    y = defuzz(acts, LIGHT_OUT, LIGHT_OUT_U)
    return (None if y is None else max(0.0, min(100.0, y))), acts

def curtain_fis(light_pct, temp, aqi, hour, prev_angle, slots, use_ext=True):
    light_pct = clamp(light_pct, 0.0, 100.0); temp = clamp(temp, 0.0, 40.0); aqi = clamp(aqi, 0.0, 500.0)
    hour = clamp(hour, 0.0, 24.0); prev_angle = clamp(prev_angle, 0.0, 75.0)
    mL = {k: mf(light_pct, p) for k, p in LIGHT_IN.items()}
    mT = {k: mf(temp, p) for k, p in TEMP_CFIS.items()}
    mA = {k: mf(aqi, p) for k, p in AQI_IN.items()}
    mS = {k: mf(hour, p) for k, p in slots.items()}
    mP = {k: mf(prev_angle, p) for k, p in ANGLE.items()}
    acts = {"Open": 0.0, "Partial": 0.0, "Closed": 0.0}
    rules = CURTAIN_RULES if use_ext else CURTAIN_RULES[:11]
    for (L, T, A, S, P, o) in rules:
        a = min(
            1.0 if L is None else mL[L],
            1.0 if T is None else mT[T],
            1.0 if A is None else mA[A],
            1.0 if S is None else mS[S],
            1.0 if P is None else mP[P],
        )
        if a > acts[o]:
            acts[o] = a
    y = defuzz(acts, ANGLE, ANGLE_U)
    if y is None:
        return prev_angle, acts, False   # hold
    return max(0.0, min(75.0, y)), acts, True

# ---------------- environment ----------------
P_FIX = 65.0
N_FIX = 12
P_DIM = 34.0
E_ART_FULL = 500.0 * P_FIX / P_DIM      # 955.882 lux per zone at 100 %
E_REF = 1000.0                          # lux that maps to 100 % LDR reading
DF = {"Window": 0.050, "Center": 0.020, "Corridor": 0.008}
ROWS = ("Window", "Center", "Corridor")
SLAT_DEPTH, SLAT_PITCH, TAU_SLAT = 0.17, 0.16, 0.05
STATIC_ANGLE = 0.0

COMFORT = (400.0, 2000.0)
PIR_HOLD_MIN = 10

WEATHER = {
    "Normal":   dict(Ediff=20000.0, Tmean=24.0, Tamp=4.0, dTgain=5.0, AQIout=45.0),
    "Heatwave": dict(Ediff=25000.0, Tmean=34.0, Tamp=4.0, dTgain=5.0, AQIout=45.0),
    "Pollution": dict(Ediff=20000.0, Tmean=24.0, Tamp=4.0, dTgain=5.0, AQIout=250.0),
}
A_WIN = 3 * 3.5 * 1.6      # 16.8 m2 glazing
SHGC = 0.60
COP = 3.0
I_PEAK = {"Normal": 400.0, "Heatwave": 550.0, "Pollution": 400.0}   # W/m2 on facade
TAU_TH_H = 1.0     # thermal time constant [h]
TAU_AQ_H = 0.5     # air time constant [h]
AQ_OCC_GAIN = 60.0

def sun_factor(t):
    return math.sin(math.pi * (t - 6.0) / 12.0) if 6.0 <= t <= 18.0 else 0.0

def coverage(theta):
    return min(1.0, SLAT_DEPTH * math.sin(math.radians(theta)) / SLAT_PITCH)

def tau_blind(theta):
    return 1.0 - coverage(theta) * (1.0 - TAU_SLAT)

def t_out(t, w):
    return w["Tmean"] + w["Tamp"] * math.cos(2 * math.pi * (t - 15.0) / 24.0)

def occupied(t):
    return (9.0 <= t < 13.0) or (14.0 <= t < 18.0)

def comfort_mu(E):
    return trap(E, 300.0, 400.0, 2000.0, 3000.0)

# ---------------- simulation ----------------
SCEN = {
    0: "Baseline (24/7 ON)",
    1: "Manager (09:00-18:00)",
    2: "PIR occupancy (10-min hold)",
    3: "Uniform dimming 34 W (24/7)",
```

```
    4: "Static daylight (window row OFF 09-13)",
    5: "Fuzzy lighting, CCIS 2024 (blinds static 45 deg)",
    6: "Fuzzy win-curtain co-regulation, IoT-J 2026",
}

def simulate(scen, weather="Normal", slots=SLOT_CLASS, step_min=1, log=False):
    lrules = LIGHT_RULES
    w = WEATHER[weather]
    n = int(24 * 60 / step_min)
    dt = step_min / 60.0
    theta = 75.0 if scen == 6 else STATIC_ANGLE
    Tin = t_out(0.0, w)  # start at equilibrium (night, S=0)
    aqi = w["AQIout"]
    last_occ = -10**9
    E_Wh = 0.0
    occ_min = comfort_min = 0
    comfort_sum = 0.0
    rows_log = []
    held = 0
    Ecool_Wh = 0.0
    util_sum = 0.0
    Tpeak = -1e9; dh28 = 0.0; prev_theta = theta; reversals = 0; last_dir = 0; travel = 0.0
    for k in range(n):
        t = k * dt
        S = sun_factor(t)
        occ = occupied(t)
        if occ:
            last_occ = k
        pir = occ or (last_occ >= 0 and (k - last_occ) * step_min <= PIR_HOLD_MIN)
        # sensors (state at start of step, uses theta from previous step)
        Ediff = w["Ediff"] * S
        E_win_unshaded = DF["Window"] * Ediff
        L_curtain = min(100.0, E_win_unshaded / E_REF * 100.0)
        # ---- curtain decision ----
        if scen == 6:
            if pir:
                theta, acts, ok = curtain_fis(L_curtain, Tin, aqi, t, theta, slots)
                if not ok:
                    held += 1
            else:
                theta = 75.0
        tau = tau_blind(theta)
        # ---- lighting decision ----
        La = {}
        Eday = {r: DF[r] * Ediff * tau for r in ROWS}
        for r in ROWS:
            if scen == 0:
                La[r] = 1.0
            elif scen == 1:
                La[r] = 1.0 if 9.0 <= t < 18.0 else 0.0
            elif scen == 2:
                La[r] = 1.0 if pir else 0.0
            elif scen == 3:
                La[r] = P_DIM / P_FIX
            elif scen == 4:
                La[r] = 0.0 if (r == "Window" and 9.0 <= t < 13.0) else 1.0
            else:
                if pir:
                    Lpct = min(100.0, Eday[r] / E_REF * 100.0)
                    y, _ = light_fis(Lpct, t, Tin, slots, lrules)
                    La[r] = (y if y is not None else 0.0) / 100.0
                else:
                    La[r] = 0.0
        P = sum(4 * P_FIX * La[r] for r in ROWS)
        E_Wh += P * dt
        if occ:
            Qs = SHGC * A_WIN * I_PEAK[weather] * S * tau
            Ecool_Wh += (Qs + P) / COP * dt
        Etask = {r: Eday[r] + E_ART_FULL * La[r] for r in ROWS}
        if occ:
            Tpeak = max(Tpeak, Tin)
            dh28 += max(0.0, Tin - 28.0) * dt
        d = theta - prev_theta
        travel += abs(d)
        if abs(d) > 0.5:
            dr = 1 if d > 0 else -1
            if last_dir and dr != last_dir: reversals += 1
            last_dir = dr
        prev_theta = theta
        if occ:
            occ_min += step_min
            c = sum(comfort_mu(Etask[r]) for r in ROWS) / 3.0
            comfort_sum += c * step_min
            util_sum += 100.0 * sum(La[r] for r in ROWS) / 3.0 * step_min
            if all(COMFORT[0] <= Etask[r] <= COMFORT[1] for r in ROWS):
                comfort_min += step_min
        if log:
            rows_log.append(dict(t=t, P=P, theta=theta, Tin=Tin, aqi=aqi, pir=pir,
                                 La={r: La[r] for r in ROWS}, Eday=Eday, Etask=Etask, Lc=L_curtain))
        # ---- physics integration (explicit Euler, after decisions) ----
        Teq = t_out(t, w) + w["dTgain"] * S * tau
        Tin += (Teq - Tin) * (dt / TAU_TH_H)
        aq_eq = w["AQIout"] + (AQ_OCC_GAIN if occ else 0.0)
        aqi += (aq_eq - aqi) * (dt / TAU_AQ_H)
    return dict(E_kWh=E_Wh / 1000.0, occ_min=occ_min, comfort_min=comfort_min,
                Fc_avg=(comfort_sum / occ_min if occ_min else 0.0), held=held, log=rows_log,
                Tpeak=Tpeak, U_avg=(util_sum / occ_min if occ_min else 0.0), Ecool_kWh=Ecool_Wh/1000.0, dh28=dh28,
```

```
»» reversals=reversals, travel=travel)

if __name__ == "__main__":
    base = simulate(0)["E_kWh"]
    for wname in ("Normal", "Heatwave", "Pollution"):
        print("==", wname)
        e2 = simulate(2, wname)["E_kWh"]
        for s in range(7):
            r = simulate(s, wname)
            print(f"S{s} {r['E_kWh']:.4f} kWh  vsS0 {100*(1-r['E_kWh']/base):6.2f}%  vsS2 {100*(1-r['E_kWh']/e2):6.2f}%  "
                  f"cool {r['Ecool_kWh']:.3f} tot {r['E_kWh']+r['Ecool_kWh']:.3f} band {100*r['comfort_min']/max(1,r[
»» 'occ_min']):5.1f}%  Fc {r['Fc_avg']:.3f} held {r['held']} Tpk {r['Tpeak']:.2f} dh28 {r['dh28']:.2f} rev {r['reversals']}
»» trav {r['travel']:.0f}")
```